

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM THÀNH PHỐ HỒ CHÍ MINH

ĐIỂM SỐ	ĐIỂM CHỮ	Cán bộ chấm thi 1 <i>(Kí, ghi rõ họ tên)</i>	Cán bộ chấm thi 2 <i>(Kí, ghi rõ họ tên)</i>

TIỂU LUẬN HỌC PHẦN
HỌC SÂU

PHÁT HIỆN BIẾN BÁO GIAO THÔNG CHO XE TỰ HÀNH:
SO SÁNH YOLO VỚI DETR VÀ KHẢO SÁT
KIẾN TRÚC NHẹ, LƯỢNG TỬ HOÁ
VÀ CHỨNG CẤT TRI THỨC

Ngành: Khoa học máy tính

Khóa: 36 (2025 – 2027)

Giảng viên giảng dạy:

Tên học viên: Huỳnh Phát Lợi

Mã số học viên: KHMT836016

Thành phố Hồ Chí Minh, tháng 08 năm 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM THÀNH PHỐ HỒ CHÍ MINH**



**TIỂU LUẬN HỌC PHẦN
HỌC SÂU**

**PHÁT HIỆN BIỂN BÁO GIAO THÔNG CHO XE TỰ HÀNH:
SO SÁNH YOLO VỚI DETR VÀ KHẢO SÁT
KIẾN TRÚC NHẸ, LƯỢNG TỬ HOÁ
VÀ CHUNG CẤT TRI THỨC**

Ngành: Khoa học máy tính

Khóa: 36 (2025 – 2027)

Giảng viên giảng dạy:

Tên học viên: Huỳnh Phát Lợi

Mã số học viên: KHMT836016

Thành phố Hồ Chí Minh, tháng 08 năm 2026

LỜI CAM ĐOAN

Tôi xin cam đoan bài tiểu luận với đề tài “PHÁT HIỆN BIỂN BÁO GIAO THÔNG CHO XE TỰ HÀNH: SO SÁNH YOLO VỚI DETR VÀ KHẢO SÁT KIẾN TRÚC NHẹ, LƯỢNG TỬ HOÁ, CHUNG CẤT TRI THỨC” là kết quả nghiên cứu và thực hiện của cá nhân tôi dưới sự hướng dẫn của giảng viên phụ trách học phần.

Các nội dung trình bày trong bài tiểu luận được tổng hợp, phân tích và diễn giải dựa trên quá trình tìm hiểu tài liệu, nghiên cứu phương pháp và thực nghiệm đánh giá. Các tài liệu, công trình nghiên cứu, dữ liệu và nguồn tham khảo được sử dụng trong bài đều được trích dẫn và ghi rõ trong phần tài liệu tham khảo.

Tôi cam đoan rằng bài tiểu luận không sao chép nguyên văn từ bất kỳ công trình nào khác. Các thực nghiệm được thực hiện trung thực, có cơ sở kiểm chứng và có thể tái lập. Mọi sai sót, nếu có, thuộc trách nhiệm của cá nhân tôi.

Thành phố Hồ Chí Minh, ngày 06 tháng 08 năm 2026

Học viên thực hiện

(Ký và ghi rõ họ tên)

Huỳnh Phát Lợi

LỜI CẢM ƠN

Trước hết, tôi xin gửi lời cảm ơn chân thành đến quý thầy cô phụ trách học phần HỌC SÂU, đã tận tình giảng dạy, định hướng và cung cấp những kiến thức nền tảng quan trọng cho đề tài này.

Những kiến thức về kiến trúc mạng nơ-ron tích chập, cơ chế chú ý và các phương pháp đánh giá mô hình được truyền đạt trong học phần đã giúp tôi có cơ sở để tiếp cận đề tài “PHÁT HIỆN BIỂN BÁO GIAO THÔNG CHO XE TỰ HÀNH: SO SÁNH YOLO VỚI DETR VÀ KHẢO SÁT KIẾN TRÚC NHẹ, LƯỢNG TỬ HOÁ, CHUNG CẤT TRI THỨC” một cách có hệ thống hơn, đặc biệt là ở khâu phân tích nguyên nhân đằng sau các con số thay vì chỉ dừng lại ở việc báo cáo kết quả.

Tôi cũng xin cảm ơn các bạn cùng nhóm đã phối hợp trong quá trình xây dựng mã nguồn và tiến hành thực nghiệm, cũng như đã trao đổi thẳng thắn về những kết quả không như kỳ vọng — những trao đổi đó đã góp phần định hình phần phân tích ở Chương 4.

Tôi xin cảm ơn TRƯỜNG ĐẠI HỌC SƯ PHẠM THÀNH PHỐ HỒ CHÍ MINH và Khoa Công nghệ Thông tin đã tạo điều kiện học tập, nghiên cứu và cung cấp môi trường học thuật thuận lợi trong quá trình thực hiện bài tiểu luận này.

Do giới hạn về thời gian, tài nguyên tính toán và kinh nghiệm nghiên cứu, bài tiểu luận khó tránh khỏi những thiếu sót; một số hạn chế đã được tôi nêu rõ ở Chương 5. Tôi rất mong nhận được những góp ý từ giảng viên để có thể tiếp tục hoàn thiện nội dung và phát triển đề tài tốt hơn trong các nghiên cứu tiếp theo.

TÓM TẮT

Phát hiện biển báo giao thông là một khối chức năng bắt buộc của hệ thống hỗ trợ lái tiên tiến và xe tự hành. Bài toán đặt ra hai ràng buộc đối nghịch: mô hình phải đủ chính xác để không bỏ sót biển báo, đồng thời phải đủ nhẹ và đủ nhanh để chạy thời gian thực trên phần cứng nhúng gắn trên xe. Trong vài năm gần đây, hai họ kiến trúc thống trị lĩnh vực phát hiện đối tượng đi theo hai triết lý khác hẳn nhau: họ một giai đoạn dựa trên mạng tích chập mà đại diện là YOLO, và họ transformer dự đoán theo tập hợp mà đại diện là DETR. Câu hỏi thực tiễn đặt ra cho người triển khai là: với một bộ dữ liệu quy mô vừa và ngân sách tính toán của một đồ án học phần, họ nào cho kết quả tốt hơn, và cái giá phải trả cho mỗi lựa chọn là gì.

Đề tài sử dụng bộ dữ liệu *pkdarabi/cardetection* trên Kaggle, gồm 4.969 ảnh độ phân giải 416×416 với 6.012 hộp bao thuộc 15 lớp là đèn tín hiệu và biển giới hạn tốc độ. Toàn bộ quy trình được xây dựng dưới dạng các mô-đun Python độc lập, có kiểm thử tự động, với sổ tay Jupyter chỉ đóng vai trò điều phối. Bốn nhóm thực nghiệm được tiến hành: so sánh mô hình nền giữa YOLOv8n và DETR-ResNet50; khảo sát ba kiến trúc nhẹ thay thế gồm YOLO11n, SSDLite-MobileNetV3 và D-FINE-Nano; lượng tử hoá sau huấn luyện theo ba đường FP16, ONNX INT8 và OpenVINO INT8; và cuối cùng là chứng cất tri thức từ YOLO26s sang YOLO26n.

Mọi mô hình được huấn luyện với cùng một hạt giống ngẫu nhiên và được đánh giá trên tập kiểm tra tách biệt. Các độ đo gồm mAP tại ngưỡng IoU 0,5 và mAP trung bình trên dải ngưỡng 0,5–0,95, kèm theo độ trễ suy luận, tốc độ khung hình và dung lượng mô hình để phản ánh ràng buộc triển khai. Một phân tích khám phá dữ liệu đầy đủ được thực hiện trước khi huấn luyện, bao gồm phân bố lớp, thống kê kích thước hộp bao, bản đồ nhiệt vị trí vật thể và kiểm tra chất lượng nhãn.

Kết quả cho thấy YOLOv8n đạt $\text{mAP}@0,5$ bằng 0,9539 và $\text{mAP}@0,5:0,95$ bằng 0,8058 trên tập kiểm tra, với 6,25 MB trọng số và độ trễ 8,83 ms mỗi ảnh. DETR-ResNet50 chỉ đạt $\text{mAP}@0,5$ bằng 0,1220 sau 10 chu kỳ huấn luyện, tức kém hơn gần tám lần, đồng thời nặng hơn 26,6 lần và chậm hơn 7,2 lần. Ba kiến trúc nhẹ ở nhóm thực nghiệm thứ hai đều không vượt được mô hình nền. Lượng tử hoá FP16 giữ nguyên độ chính xác trong khi

giảm khoảng một nửa dung lượng, còn INT8 tuy nén mạnh hơn nhưng lại chậm đi trên môi trường thử nghiệm. Chứng cất tri thức cho kết quả *âm*: mô hình học trò được chứng cất đạt 0,9014, thấp hơn chính nó khi huấn luyện thông thường (0,9348).

Đóng góp của đề tài không nằm ở một con số mAP cao mà ở hai phát hiện mang tính phương pháp luận. Thứ nhất, khoảng cách giữa DETR và YOLO trong điều kiện này chủ yếu do điều kiện huấn luyện bất đối xứng chứ không phải do bản chất kiến trúc, và báo cáo phân tách rõ từng yếu tố gây nhiễu. Thứ hai, phân tích thành phần bộ dữ liệu chỉ ra rằng 38,7% số ảnh huấn luyện là ảnh cắt cận cảnh có cạnh biển chiếm tới 66,8% cạnh ảnh, khiến phân bố kích thước vật thể bị lưỡng cực và giải thích vì sao mô hình đạt mAP rất cao trên tập kiểm tra nhưng suy giảm rõ rệt trên video đường phố thực tế. Hạn chế lớn nhất là hai nhánh mô hình dùng hai thư viện tính mAP khác nhau và bộ dữ liệu tồn tại hiện tượng trùng ảnh giữa các tập.

Từ khoá: phát hiện biển báo giao thông, YOLO, DETR, lượng tử hoá, chứng cất tri thức, dịch chuyển miền, vật thể nhỏ.

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Diễn giải
ADAS	Advanced Driver Assistance System – Hệ thống hỗ trợ lái tiên tiến
AdamW	Adam with decoupled Weight decay – Thuật toán tối ưu Adam có tách rời suy giảm trọng số
AP	Average Precision – Độ chính xác trung bình
API	Application Programming Interface – Giao diện lập trình ứng dụng
BCE	Binary Cross-Entropy – Hàm mất mát entropy chéo nhị phân
CIoU	Complete Intersection over Union – Độ đo IoU mở rộng dùng cho hồi quy hộp bao
CNN	Convolutional Neural Network – Mạng nơ-ron tích chập
COCO	Common Objects in Context – Bộ dữ liệu và định dạng chú thích chuẩn
CSV	Comma-Separated Values – Định dạng dữ liệu phân tách bằng dấu phẩy
DETR	DEtection TRansformer – Mô hình phát hiện đối tượng dựa trên transformer
DFL	Distribution Focal Loss – Hàm mất mát tiêu điểm dạng phân phối
FN	False Negative – Trường hợp âm tính giả (bỏ sót)
FP	False Positive – Trường hợp dương tính giả (báo nhầm)
FP16	Floating Point 16-bit – Số thực dấu phẩy động 16 bit

Từ viết tắt	Diễn giải
FPN	Feature Pyramid Network – Mạng kim tự tháp đặc trưng
FPS	Frames Per Second – Số khung hình xử lý được mỗi giây
GFLOPs	Giga Floating Point Operations – Tỷ phép tính dấu phẩy động
GIoU	Generalized Intersection over Union – Độ đo IoU tổng quát hoá
GPU	Graphics Processing Unit – Bộ xử lý đồ hoạ
GTSDb	German Traffic Sign Detection Benchmark – Bộ chuẩn phát hiện biển báo của Đức
GTSRB	German Traffic Sign Recognition Benchmark – Bộ chuẩn nhận dạng biển báo của Đức
INT8	Integer 8-bit – Số nguyên 8 bit
IoU	Intersection over Union – Tỷ số giao trên hợp giữa hai hộp bao
JSON	JavaScript Object Notation – Định dạng trao đổi dữ liệu dạng văn bản
LR	Learning Rate – Tốc độ học
mAP	mean Average Precision – Độ chính xác trung bình lấy trung bình trên các lớp
NMS	Non-Maximum Suppression – Kỹ thuật loại bỏ hộp bao trùng lặp
ONNX	Open Neural Network Exchange – Định dạng trao đổi mô hình mạng nơ-ron
PR	Precision–Recall – Đường cong độ chính xác theo độ bao phủ

Từ viết tắt	Diễn giải
PTQ	Post-Training Quantization – Lượng tử hoá sau huấn luyện
RGB	Red – Green – Blue – Không gian màu ba kênh
SSD	Single Shot MultiBox Detector – Bộ phát hiện đơn lượt
TP	True Positive – Trường hợp dương tính đúng
VOC	Visual Object Classes – Bộ dữ liệu và định dạng chú thích PASCAL VOC
VRAM	Video Random Access Memory – Bộ nhớ của card đồ hoạ
YOLO	You Only Look Once – Họ mô hình phát hiện đối tượng một giai đoạn

DANH MỤC BẢNG BIỂU

3.1	Thống kê tổng quan bộ dữ liệu theo từng tập	11
3.2	Phân bố lớp chi tiết, sắp theo số hộp bao giảm dần	13
3.3	Phân nhóm kích thước hộp bao theo diện tích chuẩn hoá	14
3.4	Thành phần bộ dữ liệu theo nguồn gốc suy ra từ quy ước đặt tên tệp	15
3.5	Phân vị diện tích hộp bao và cạnh tương ứng trên ảnh 416×416	15
3.6	Tổ chức mô-đun của quy trình	17
3.7	Thiết kế bốn nhóm thực nghiệm	18
3.8	Siêu tham số huấn luyện YOLOv8n	19
3.9	Siêu tham số huấn luyện DETR-ResNet50	20
3.10	Siêu tham số nhóm E3	21
4.1	Tổng hợp kết quả bốn nhóm thực nghiệm trên tập kiểm tra	24
4.2	So sánh chi tiết YOLOv8n và DETR-ResNet50 trên tập kiểm tra	25
4.3	Ảnh hưởng của lượng tử hoá tới độ chính xác và tốc độ (toàn bộ thuộc giai đoạn 2)	28
4.4	Kết quả nhóm E3 đo bằng pycocotools trên tập kiểm tra	29
4.5	Ba phép so sánh cốt lõi của nhóm E3	30
4.6	Kiểm tra các ngưỡng chấp nhận khai báo trước khi chạy thực nghiệm	31
.1	Tài nguyên sử dụng trong đề tài	44
.2	Danh sách 15 lớp và mã tương ứng	44
.3	Ảnh xạ giữa nội dung báo cáo và tệp kết quả sinh ra bởi quy trình	45

DANH MỤC HÌNH ẢNH

3.1	Phân bố số hộp bao theo từng lớp trên toàn bộ dữ liệu	12
3.2	Thống kê kích thước hộp bao trên toàn bộ dữ liệu	14
3.3	Ba chế độ ảnh cùng tồn tại trong bộ dữ liệu, kèm hộp bao nhãn thật	16
3.4	Bản đồ nhiệt vị trí tâm vật thể trên toàn bộ dữ liệu	16
4.1	Diễn biến hàm mất mát và các chỉ số của YOLOv8n qua 30 chu kỳ huấn luyện	25
4.2	Đặc tính phân loại của YOLOv8n trên tập kiểm định	26
4.3	Ma trận nhầm lẫn chuẩn hoá của YOLOv8n trên tập kiểm định	33
4.4	Đối chiếu nhãn thật và dự đoán trên một lô ảnh kiểm định	34

MỤC LỤC

1	GIỚI THIỆU TỔNG QUAN	1
1.1	Đặt vấn đề	1
1.2	Mục tiêu đề tài	2
1.3	Đối tượng và phạm vi nghiên cứu	3
1.4	Đóng góp của đề tài	3
1.5	Bố cục báo cáo	4
2	CƠ SỞ LÝ THUYẾT VÀ CÔNG TRÌNH LIÊN QUAN	5
2.1	Bài toán phát hiện đối tượng	5
2.1.1	Phát biểu bài toán	5
2.1.2	Định dạng chú thích	5
2.2	Họ kiến trúc một giai đoạn	6
2.2.1	Nguyên lý chung	6
2.2.2	Đa tỉ lệ và vật thể nhỏ	6
2.2.3	YOLOv8 và các hàm mất mát	6
2.3	Họ kiến trúc transformer	7
2.3.1	Cơ chế chú ý	7
2.3.2	DETR: dự đoán theo tập hợp	7
2.4	Hệ thống độ đo	8
2.4.1	IoU, Precision và Recall	8
2.4.2	mAP	8
2.4.3	Độ đo chi phí triển khai	8
2.5	Nén mô hình	9
2.5.1	Lượng tử hoá	9
2.5.2	Chứng cất tri thức	9
2.6	Công trình liên quan về biển báo giao thông	9
3	DỮ LIỆU VÀ PHƯƠNG PHÁP	11
3.1	Bộ dữ liệu	11

3.1.1	Nguồn và quy mô	11
3.1.2	Phân bố lớp và mất cân bằng	12
3.1.3	Thông kê kích thước hộp bao	14
3.1.4	Phân tích thành phần bộ dữ liệu	14
3.1.5	Vị trí vật thể trong khung hình	16
3.1.6	Kiểm tra chất lượng nhãn	17
3.2	Kiến trúc quy trình	17
3.2.1	Bảo đảm tính tái lập	17
3.3	Thiết kế thực nghiệm	18
3.3.1	Cấu hình huấn luyện nhánh YOLO	18
3.3.2	Cấu hình huấn luyện nhánh DETR	19
3.3.3	Cấu hình nhóm E3 — chứng cất tri thức	20
3.3.4	Kiểm toán rò rỉ dữ liệu giữa các tập	21
3.3.5	Quy trình đánh giá	22
4	KẾT QUẢ VÀ BÀN LUẬN	23
4.1	Kết quả tổng quan	23
4.1.1	Hai giai đoạn thực nghiệm	23
4.1.2	Bảng tổng hợp	24
4.2	E0 — So sánh mô hình nền	25
4.2.1	Kết quả chi tiết	25
4.2.2	Vì sao DETR kém hơn tới gần tám lần	26
4.3	E1 — Khảo sát kiến trúc nhẹ	27
4.4	E2 — Lượng tử hoá sau huấn luyện	28
4.5	Kết quả trái với kỳ vọng	29
4.5.1	E3 — Chứng cất tri thức cho kết quả âm	29
4.5.2	Khoảng cách giữa chỉ số và hành vi thực tế	31
4.6	Phân tích lỗi	33
4.6.1	Nhầm lẫn giữa các lớp	33
4.6.2	Ảnh hưởng của mất cân bằng lớp	34
4.7	Bàn luận	34

5	TỔNG KẾT	36
5.1	Tóm lược kết quả đạt được	36
5.2	Đóng góp	36
5.3	Hạn chế	37
5.4	Hướng phát triển	38
	TÀI LIỆU THAM KHẢO	40
	PHỤ LỤC	44
A.	Tài nguyên công khai	44
B.	Danh sách lớp	44
C.	Cách chạy lại quy trình	44
D.	Bảng ánh xạ hình vẽ và nguồn dữ liệu	45
E.	Ghi chú về tính tái lập	46

CHƯƠNG 1 : GIỚI THIỆU TỔNG QUAN

1.1 Đặt vấn đề

Xe tự hành và các hệ thống hỗ trợ lái tiên tiến (ADAS) phải hiểu được môi trường giao thông xung quanh trước khi có thể ra bất kỳ quyết định điều khiển nào. Trong số các thành phần của bài toán nhận thức đó, phát hiện biển báo giao thông giữ một vị trí đặc biệt: biển báo là kênh giao tiếp chính thức giữa hạ tầng đường bộ và người lái, và nội dung của nó ràng buộc trực tiếp hành vi hợp pháp của phương tiện. Một hệ thống bỏ sót biển giới hạn tốc độ hoặc đọc nhầm đèn tín hiệu không chỉ gây khó chịu mà có thể dẫn tới hậu quả về an toàn và pháp lý.

Về mặt kỹ thuật, đây là bài toán phát hiện đối tượng: với mỗi ảnh đầu vào, mô hình phải đồng thời *định vị* từng biển báo bằng một hộp bao và *phân loại* nó vào đúng chủng loại. Điều này khó hơn hẳn bài toán phân lớp ảnh, nơi đầu ra chỉ là một nhãn duy nhất cho cả bức ảnh. Lịch sử phát triển của lĩnh vực đi qua hai dòng lớn. Dòng hai giai đoạn, tiêu biểu là Faster R-CNN [1], tách bài toán thành bước đề xuất vùng rồi mới phân loại và tinh chỉnh hộp bao; độ chính xác cao nhưng chi phí tính toán lớn. Dòng một giai đoạn, khởi đầu từ YOLO [2] và SSD [3], dự đoán trực tiếp hộp bao và lớp trên bản đồ đặc trưng chỉ trong một lượt truyền xuôi, đổi một phần độ chính xác lấy tốc độ đủ cho ứng dụng thời gian thực.

Năm 2020, DETR [4] mở ra một hướng thứ ba bằng cách áp dụng kiến trúc transformer [5] vào phát hiện đối tượng. DETR loại bỏ hoàn toàn hộp neo và bước hậu xử lý NMS, thay bằng một tập truy vấn đối tượng học được và phép gán song ánh Hungarian [6] giữa dự đoán và nhãn thật. Thiết kế này thanh lịch về mặt khái niệm, nhưng kèm theo một cái giá ít được nhấn mạnh trong các bài trình bày phổ thông: DETR hội tụ rất chậm, bản gốc cần khoảng 300 đến 500 chu kỳ huấn luyện trên bộ dữ liệu COCO quy mô lớn [7]. Đây chính là điểm mà một đề án học phần với ngân sách GPU hạn chế sẽ va phải.

Các nút thắt chính mà đề tài phải đối mặt:

1. **Ràng buộc kép giữa độ chính xác và chi phí triển khai.** Mô hình phải chạy được trên phần cứng nhúng gắn trên xe, nơi dung lượng bộ nhớ và ngân sách năng lượng đều bị giới hạn, nhưng vẫn phải đủ chính xác để không bỏ sót biển báo.
2. **Dữ liệu quy mô vừa và mất cân bằng.** Bộ dữ liệu sử dụng chỉ có 6.012 hộp bao cho

15 lớp, với tỉ số mất cân bằng giữa lớp phổ biến nhất và lớp hiếm nhất lên tới 35,77 lần. Mất cân bằng lớp là một vấn đề đã được ghi nhận rộng rãi trong học sâu [8] và ảnh hưởng trực tiếp tới các lớp thiểu số.

3. **Vật thể nhỏ.** Biển báo ở xa chỉ chiếm vài chục điểm ảnh trong khung hình. Phát hiện vật thể nhỏ là điểm yếu cố hữu của các bộ phát hiện hiện đại [9], và mức độ nghiêm trọng của nó phụ thuộc mạnh vào phân bố kích thước trong dữ liệu huấn luyện.
4. **So sánh công bằng giữa hai hệ sinh thái khác nhau.** YOLO và DETR đến từ hai thư viện khác nhau, dùng hai định dạng nhãn khác nhau và hai cách tính mAP khác nhau. Nếu không kiểm soát, phần chênh lệch do công cụ đo sẽ lẫn vào phần chênh lệch do mô hình.

Câu hỏi nghiên cứu 1.1 *Với một bộ dữ liệu biển báo quy mô vừa và ngân sách huấn luyện hạn chế, kiến trúc một giai đoạn dựa trên tích chập và kiến trúc transformer dự đoán theo tập hợp cho kết quả khác nhau như thế nào về độ chính xác, tốc độ và dung lượng?*

Câu hỏi nghiên cứu 1.2 *Có kiến trúc nhẹ nào thay thế được mô hình nền mà không đánh đổi độ chính xác, và các kỹ thuật nén mô hình như lượng tử hoá hay chưng cất tri thức mang lại lợi ích gì trong bối cảnh này?*

Câu hỏi nghiên cứu 1.3 *Chỉ số mAP đo trên tập kiểm tra phản ánh đến mức nào năng lực thực tế của mô hình khi gặp cảnh đường phố thật, và đặc điểm nào của bộ dữ liệu giải thích khoảng cách đó?*

1.2 Mục tiêu đề tài

Đề tài hướng đến các mục tiêu chính sau:

- Xây dựng một quy trình xử lý hoàn chỉnh và tái lập được, đi từ dữ liệu thô tới mô hình đã huấn luyện và bảng số liệu đánh giá, trong đó mọi bước đều tồn tại dưới dạng mô-đun độc lập chạy được từ dòng lệnh và có kiểm thử tự động.
- Thiết lập một mô hình nền đáng tin cậy bằng YOLOv8n và một mô hình đối chứng bằng DETR-ResNet50, đo trên cùng một tập kiểm tra với cùng một hạt giống ngẫu nhiên.

- Khảo sát ba kiến trúc nhẹ thay thế và hai nhóm kỹ thuật nén mô hình, nhằm trả lời câu hỏi liệu có thể giảm chi phí triển khai mà không mất độ chính xác.
- Phân tích khám phá dữ liệu ở mức đủ sâu để giải thích được kết quả, thay vì chỉ báo cáo con số cuối cùng.
- Báo cáo trung thực cả những kết quả đi ngược giả thuyết ban đầu, kèm theo phân tích nguyên nhân.

1.3 Đối tượng và phạm vi nghiên cứu

Đối tượng nghiên cứu: các mô hình học sâu phát hiện đối tượng áp dụng cho biển báo giao thông, xét đồng thời trên ba trục độ chính xác, tốc độ suy luận và dung lượng mô hình.

Phạm vi nghiên cứu: đề tài giới hạn ở bộ dữ liệu *pkdarabi/cardetection* với 15 lớp gốc, giữ nguyên không ánh xạ lại nhãn. Việc huấn luyện được thực hiện trên môi trường Google Colab với GPU đơn, ngân sách 30 chu kỳ cho nhánh YOLO và 10 chu kỳ cho nhánh DETR. Đề tài *không* bao gồm: thu thập và gán nhãn dữ liệu mới, tinh chỉnh siêu tham số bằng tìm kiếm tự động, huấn luyện DETR tới hội tụ hoàn toàn, và triển khai thực tế trên phần cứng nhúng. Những giới hạn này được nêu rõ ngay từ đầu vì chúng ảnh hưởng trực tiếp tới cách diễn giải kết quả ở Chương 4.

1.4 Đóng góp của đề tài

1. Một quy trình tái lập được cho bài toán phát hiện biển báo, trong đó toàn bộ bộ sinh số ngẫu nhiên được gieo hạt tập trung, mọi đường dẫn được quản lý ở một nơi duy nhất, và bộ kiểm thử tự động gồm 43 trường hợp chạy trên CPU trong khoảng một giây.
2. Một bảng so sánh bốn nhóm thực nghiệm trên cùng một tập kiểm tra: mô hình nền, kiến trúc nhẹ, lượng tử hoá và chưng cất tri thức — cho phép đọc được sự đánh đổi giữa ba trục thay vì chỉ một con số mAP.
3. Một phân tích thành phần bộ dữ liệu chỉ ra rằng 38,7% số ảnh là ảnh cắt cận cảnh kiểu GTSRB [10], khiến phân bố kích thước vật thể trở nên lưỡng cực. Phát hiện này giải thích được khoảng cách giữa chỉ số trên tập kiểm tra và hành vi quan sát được trên video đường phố thực tế, và nó liên hệ trực tiếp tới hiện tượng thiên lệch bộ dữ liệu đã được mô tả trong tài liệu [11].

4. Hai kết quả âm được báo cáo đầy đủ thay vì lược bỏ: chứng cất tri thức làm giảm độ chính xác so với đối chứng, và lượng tử hoá INT8 làm giảm tốc độ thay vì tăng trên môi trường thử nghiệm.

1.5 Bố cục báo cáo

Chương 1 giới thiệu tổng quan về bài toán, mục tiêu và phạm vi. Chương 2 trình bày cơ sở lý thuyết về các họ kiến trúc phát hiện đối tượng, các hàm mất mát và hệ thống độ đo. Chương 3 mô tả bộ dữ liệu, kết quả phân tích khám phá, kiến trúc quy trình và thiết kế của bốn nhóm thực nghiệm. Chương 4 trình bày và bàn luận kết quả, bao gồm phần phân tích lỗi và các kết quả trái với kỳ vọng. Chương 5 tổng kết, nêu hạn chế và hướng phát triển.

CHƯƠNG 2 : CƠ SỞ LÝ THUYẾT VÀ CÔNG TRÌNH LIÊN QUAN

2.1 Bài toán phát hiện đối tượng

2.1.1 Phát biểu bài toán

Cho một ảnh $I \in \mathbb{R}^{H \times W \times 3}$, bài toán phát hiện đối tượng yêu cầu sinh ra một tập hợp

$$\mathcal{D} = \{(c_i, b_i, s_i)\}_{i=1}^N, \quad (2.1)$$

trong đó $c_i \in \{1, \dots, K\}$ là nhãn lớp, $b_i = (x_i, y_i, w_i, h_i)$ là hộp bao và $s_i \in [0, 1]$ là điểm tin cậy. Số lượng vật thể N không biết trước và thay đổi theo từng ảnh — đây chính là điểm khiến bài toán khó hơn phân lớp ảnh, nơi đầu ra luôn là một véc-tơ có kích thước cố định.

Sự khác biệt về bản chất này dẫn tới hai chiến lược thiết kế. Chiến lược thứ nhất là biến bài toán số lượng thay đổi thành bài toán số lượng cố định bằng cách rải dày các vị trí ứng viên trên ảnh — đó là con đường của các bộ phát hiện dựa trên hộp neo và lưới. Chiến lược thứ hai là giữ nguyên bản chất tập hợp và học một phép gán — đó là con đường của DETR.

2.1.2 Định dạng chú thích

Ba định dạng chú thích phổ biến được dùng trong đề tài này. Định dạng YOLO lưu mỗi vật thể trên một dòng văn bản gồm `class_id`, tâm hộp (x_c, y_c) và kích thước (w, h) , tất cả đã chuẩn hoá về đoạn $[0, 1]$ theo kích thước ảnh. Định dạng COCO [7] dùng một tệp JSON duy nhất chứa ba khối `images`, `annotations` và `categories`, với hộp bao lưu theo góc trên trái và kích thước tuyệt đối tính bằng điểm ảnh. Định dạng PASCAL VOC [12] dùng tệp XML riêng cho mỗi ảnh.

Phép chuyển từ YOLO sang COCO là một bước bắt buộc trong đề tài vì nhánh DETR làm việc theo lược đồ COCO. Công thức chuyển đổi:

$$\begin{aligned} x_{\text{trái}} &= \left(x_c - \frac{w}{2}\right) \cdot W_{\text{ảnh}}, & y_{\text{trên}} &= \left(y_c - \frac{h}{2}\right) \cdot H_{\text{ảnh}}, \\ w_{\text{tuyệt đối}} &= w \cdot W_{\text{ảnh}}, & h_{\text{tuyệt đối}} &= h \cdot H_{\text{ảnh}}. \end{aligned} \quad (2.2)$$

Trường `area` bằng tích $w_{\text{tuyệt đối}} \cdot h_{\text{tuyệt đối}}$ và được bộ đánh giá COCO dùng để phân nhóm vật thể thành nhỏ, trung bình và lớn khi tính mAP theo kích thước.

2.2 Họ kiến trúc một giai đoạn

2.2.1 Nguyên lý chung

Bộ phát hiện một giai đoạn bỏ hẳn bước đề xuất vùng. Ảnh đi qua một mạng xương sống tích chập để sinh bản đồ đặc trưng, sau đó một đầu dự đoán gắn trực tiếp lên bản đồ đó sẽ xuất ra hộp bao và điểm phân lớp cho từng vị trí. Vì toàn bộ quá trình chỉ cần một lượt truyền xuôi, tốc độ suy luận cao hơn đáng kể so với dòng hai giai đoạn [2].

Điểm yếu cố hữu của cách làm này là mất cân bằng cực đoan giữa số vị trí nền và số vị trí chứa vật thể. Focal Loss [13] giải quyết bằng cách giảm trọng số đóng góp của các mẫu dễ:

$$\mathcal{L}_{\text{focal}}(p_t) = -\alpha_t (1 - p_t)^\gamma \log(p_t), \quad (2.3)$$

trong đó p_t là xác suất mô hình gán cho lớp đúng và γ điều khiển mức độ giảm trọng số.

2.2.2 Đa tỉ lệ và vật thể nhỏ

Một biển báo cách xe 50 mét và một biển báo cách 5 mét có kích thước trên ảnh chênh nhau hàng chục lần. Mạng kim tự tháp đặc trưng (FPN) [14] xử lý điều này bằng cách kết hợp đặc trưng ở nhiều tầng phân giải: tầng nông giữ chi tiết không gian tốt cho vật thể nhỏ, tầng sâu giàu ngữ nghĩa hơn cho vật thể lớn.

Dù vậy, vật thể nhỏ vẫn là điểm yếu dai dẳng [9]. Nguyên nhân nằm ở bước sải: đầu dự đoán P3 của YOLOv8 hoạt động ở bước sải 8, nghĩa là một vật thể 16 điểm ảnh chỉ trải trên hai ô lưới. Khi ảnh đầu vào bị thu nhỏ để khớp kích thước huấn luyện, tình hình còn xấu hơn. Đây là lý do vì sao chỉ số AP tách riêng cho nhóm vật thể nhỏ có giá trị chẩn đoán cao hơn hẳn mAP tổng, như sẽ thấy ở Chương 4.

2.2.3 YOLOv8 và các hàm mất mát

YOLOv8 [15] là một bộ phát hiện không dùng hộp neo. Hàm mất mát tổng gồm ba thành phần:

$$\mathcal{L} = \lambda_{\text{box}} \mathcal{L}_{\text{CIoU}} + \lambda_{\text{cls}} \mathcal{L}_{\text{BCE}} + \lambda_{\text{dfl}} \mathcal{L}_{\text{DFL}}. \quad (2.4)$$

Thành phần thứ nhất dùng CIoU [16], một mở rộng của IoU có tính thêm khoảng cách giữa hai tâm hộp và độ lệch tỉ lệ khung, nhờ đó vẫn cung cấp gradient hữu ích ngay cả khi hai hộp chưa giao nhau — trường hợp mà IoU thuần túy cho gradient bằng không. Thành phần thứ ba là Distribution Focal Loss [17], coi mỗi cạnh hộp bao là một phân phối rời rạc thay vì một giá trị điểm, giúp việc hồi quy biên ổn định hơn khi ranh giới vật thể mờ.

Sau khi suy luận, bước hậu xử lý NMS loại bỏ các hộp trùng cho cùng một vật thể: các hộp được sắp theo điểm tin cậy giảm dần, hộp có điểm cao nhất được giữ lại và mọi hộp còn lại có IoU với nó vượt ngưỡng sẽ bị loại.

2.3 Họ kiến trúc transformer

2.3.1 Cơ chế chú ý

Transformer [5] thay thế phép tích chập cục bộ bằng cơ chế chú ý cho phép mọi vị trí tương tác trực tiếp với mọi vị trí khác:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \quad (2.5)$$

Hệ số $\sqrt{d_k}$ giữ cho phương sai của tích vô hướng không tăng theo số chiều, tránh đẩy hàm softmax vào vùng bão hoà. Vision Transformer [18] chứng minh kiến trúc này áp dụng được cho ảnh, nhưng cũng cho thấy nó cần lượng dữ liệu lớn hơn hẳn CNN — một quan sát rất quan trọng đối với đề tài này, vì nó dự báo trước điều gì sẽ xảy ra khi huấn luyện DETR trên bộ dữ liệu chỉ 6.012 hộp bao.

Lý do của hiện tượng đó là *thiên kiến quy nạp*. Mạng tích chập được cài sẵn giả định về tính địa phương và bất biến tịnh tiến; transformer không có những giả định này và phải học chúng từ dữ liệu. Khi dữ liệu dồi dào, việc không bị ràng buộc là lợi thế; khi dữ liệu khan hiếm, đó là bất lợi.

2.3.2 DETR: dự đoán theo tập hợp

DETR [4] đưa ra ba thay đổi so với các bộ phát hiện truyền thống. Thứ nhất, mô hình dùng một tập cố định gồm 100 truy vấn đối tượng học được, mỗi truy vấn đề xuất tối đa một vật thể. Thứ hai, việc ghép cặp giữa dự đoán và nhãn thật được giải bằng thuật toán Hungarian [6] với chi phí ghép cặp

$$\mathcal{L}_{\text{ghép}} = -\mathbb{K}_{\{c_i \neq \emptyset\}} \hat{p}_{\sigma(i)}(c_i) + \mathbb{K}_{\{c_i \neq \emptyset\}} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\sigma(i)}), \quad (2.6)$$

trong đó $\mathcal{L}_{\text{box}}$ kết hợp khoảng cách L_1 và GIoU. Thứ ba, vì phép ghép là song ánh nên mỗi nhãn thật khớp đúng một dự đoán, do đó mô hình không sinh hộp trùng và *không cần NMS*.

Cái giá của thiết kế này là tốc độ hội tụ. Bản gốc cần khoảng 300 đến 500 chu kỳ trên COCO. Nguyên nhân được cho là phép gán Hungarian thay đổi mạnh giữa các bước đầu huấn luyện, khiến tín hiệu giám sát không ổn định. Deformable DETR [19] khắc phục bằng

cơ chế chú ý chỉ lấy mẫu một số ít điểm tham chiếu, giảm số chu kỳ cần thiết xuống khoảng một phần mười. Gần đây hơn, RT-DETR [20] cho thấy dòng transformer có thể đạt tốc độ thời gian thực và vượt YOLO ở cùng ngân sách — nhưng vẫn với điều kiện được huấn luyện đầy đủ.

2.4 Hệ thống độ đo

2.4.1 IoU, Precision và Recall

Độ đo nền tảng là tỉ số giao trên hợp giữa hộp dự đoán B_p và hộp thật B_g :

$$\text{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}. \quad (2.7)$$

Một dự đoán được tính là dương tính đúng khi lớp khớp và IoU vượt ngưỡng cho trước. Từ đó suy ra

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}. \quad (2.8)$$

2.4.2 mAP

Sắp toàn bộ dự đoán theo điểm tin cậy giảm dần và tính cặp (Precision, Recall) tại từng ngưỡng cho ta đường cong PR. Độ chính xác trung bình AP là diện tích dưới đường cong này, và mAP là trung bình của AP trên toàn bộ các lớp:

$$AP = \int_0^1 p(r) dr, \quad mAP = \frac{1}{K} \sum_{k=1}^K AP_k. \quad (2.9)$$

Quy ước COCO báo cáo hai biến thể: $mAP@0,5$ tính tại một ngưỡng IoU duy nhất là 0,5, và $mAP@0,5:0,95$ lấy trung bình trên mười ngưỡng từ 0,5 đến 0,95 với bước 0,05. Biến thể thứ hai khắt khe hơn nhiều vì nó đòi hỏi hộp bao phải khít, chứ không chỉ chồng lấn ở mức tối thiểu.

Một hệ quả quan trọng của việc mAP là độ đo *xếp hạng*: khi tính mAP , không được lọc dự đoán theo ngưỡng tin cậy trước, vì làm vậy sẽ cắt mất phần đuôi của đường cong PR và bóp méo kết quả. Ngưỡng tin cậy chỉ nên áp dụng ở khâu hiển thị.

2.4.3 Độ đo chi phí triển khai

Ba đại lượng bổ sung được dùng để phản ánh ràng buộc thực tế: độ trễ suy luận tính bằng mili-giây cho mỗi ảnh, tốc độ khung hình bằng nghịch đảo của độ trễ, và dung lượng mô

hình tính bằng megabyte. Cả ba đều phụ thuộc phần cứng nên phải luôn báo cáo kèm thiết bị đo. Ngoài ra, do GPU thực thi bất đồng bộ, phép đo thời gian phải có bước đồng bộ hoá tường minh, nếu không sẽ ghi nhận thiếu thời gian tính toán thực.

2.5 Nén mô hình

2.5.1 Lượng tử hoá

Lượng tử hoá thay biểu diễn dấu phẩy động 32 bit bằng biểu diễn có độ rộng bit nhỏ hơn. Phép ánh xạ affine sang số nguyên 8 bit [21] có dạng

$$r \approx S(q - Z), \quad (2.10)$$

trong đó r là giá trị thực, q là giá trị nguyên, S là hệ số tỉ lệ và Z là điểm không. Ở mức FP16, việc giảm độ rộng bit gần như không ảnh hưởng tới chất lượng mô hình trong nhiều tác vụ [22], trong khi INT8 nén mạnh hơn nhưng có thể mất độ chính xác và — điều thường bị bỏ qua — chỉ thực sự nhanh hơn nếu môi trường thực thi có nhân tính toán số nguyên được tối ưu.

2.5.2 Chứng cất tri thức

Chứng cất tri thức [23] huấn luyện một mô hình học trò nhỏ bắt chước phân phối đầu ra mềm của một mô hình thầy lớn hơn. Hàm mất mát kết hợp

$$\mathcal{L} = (1 - \alpha) \mathcal{L}_{\text{cứng}} + \alpha T^2 \mathcal{L}_{\text{KL}}(\sigma(z_s/T) \parallel \sigma(z_t/T)), \quad (2.11)$$

với T là nhiệt độ làm mềm phân phối và z_s, z_t là logit của học trò và thầy. Giả thuyết nền là phân phối mềm của thầy mang thông tin về quan hệ giữa các lớp mà nhãn cứng không có. Tuy nhiên lợi ích này không được bảo đảm: khi khoảng cách năng lực giữa thầy và trò quá nhỏ, hoặc khi bản thân thầy chưa đủ tốt, tín hiệu chứng cất có thể trở thành nhiễu.

2.6 Công trình liên quan về biển báo giao thông

GTSRB [10] là bộ chuẩn kinh điển cho *nhận dạng* biển báo, gồm các ảnh đã cắt sát quanh từng biển; đây là bài toán phân lớp chứ không phải phát hiện. GTSDb [24] bổ sung phần *phát hiện* với ảnh đường phố đầy đủ. TT100K [25] mở rộng quy mô sang bối cảnh Trung Quốc với ảnh toàn cảnh độ phân giải cao, còn Mapillary Traffic Sign [26] cung cấp dữ liệu đa quốc gia thu từ camera hành trình.

Phân biệt giữa bộ dữ liệu kiểu nhận dạng và bộ dữ liệu kiểu phát hiện có ý nghĩa trực

tiếp với đề tài này: như sẽ trình bày ở Mục 3.1.4, bộ dữ liệu được dùng là một tập trộn chứa cả hai kiểu, và chính điều đó chi phối cách diễn giải kết quả. Hiện tượng bộ dữ liệu mang thiên lệch đặc thù và mô hình học phải thiên lệch đó thay vì học đặc trưng tổng quát đã được chỉ ra từ lâu [11], còn việc chỉ số trên tập kiểm tra không chuyển được sang dữ liệu mới cũng đã được ghi nhận trên các bộ chuẩn lớn [27].

CHƯƠNG 3 : DỮ LIỆU VÀ PHƯƠNG PHÁP

3.1 Bộ dữ liệu

3.1.1 Nguồn và quy mô

Đề tài sử dụng bộ dữ liệu *pkdarabi/cardetection* công bố trên Kaggle, phân phối dưới dạng một bản xuất Roboflow theo bố cục thư mục chuẩn `{train,valid,test}/{images,labels}` với nhãn ở định dạng YOLO TXT. Mặc dù tên trên đường dẫn là “car detection”, nội dung thực tế của toàn bộ 15 lớp đều là đèn tín hiệu và biển giới hạn tốc độ, không có lớp nào là phương tiện. Nhóm thực hiện giữ nguyên 15 lớp gốc, không gộp cũng không ánh xạ lại nhãn, để kết quả có thể đối chiếu với các công bố khác dùng cùng bộ dữ liệu.

Một quyết định thiết kế đáng nói ở đây: mô-đun kiểm tra dữ liệu được viết để *tự phát hiện* định dạng chú thích thay vì giả định sẵn. Hàm dò tìm sẽ lần lượt kiểm tra sự tồn tại của tệp `.txt` kèm `data.yaml`, rồi tới `.json`, `.xml` và `.csv` để kết luận bộ dữ liệu đang ở định dạng YOLO, COCO, VOC hay CSV. Cách làm này bảo đảm báo cáo mô tả đúng dữ liệu thật chứ không mô tả giả định của người viết.

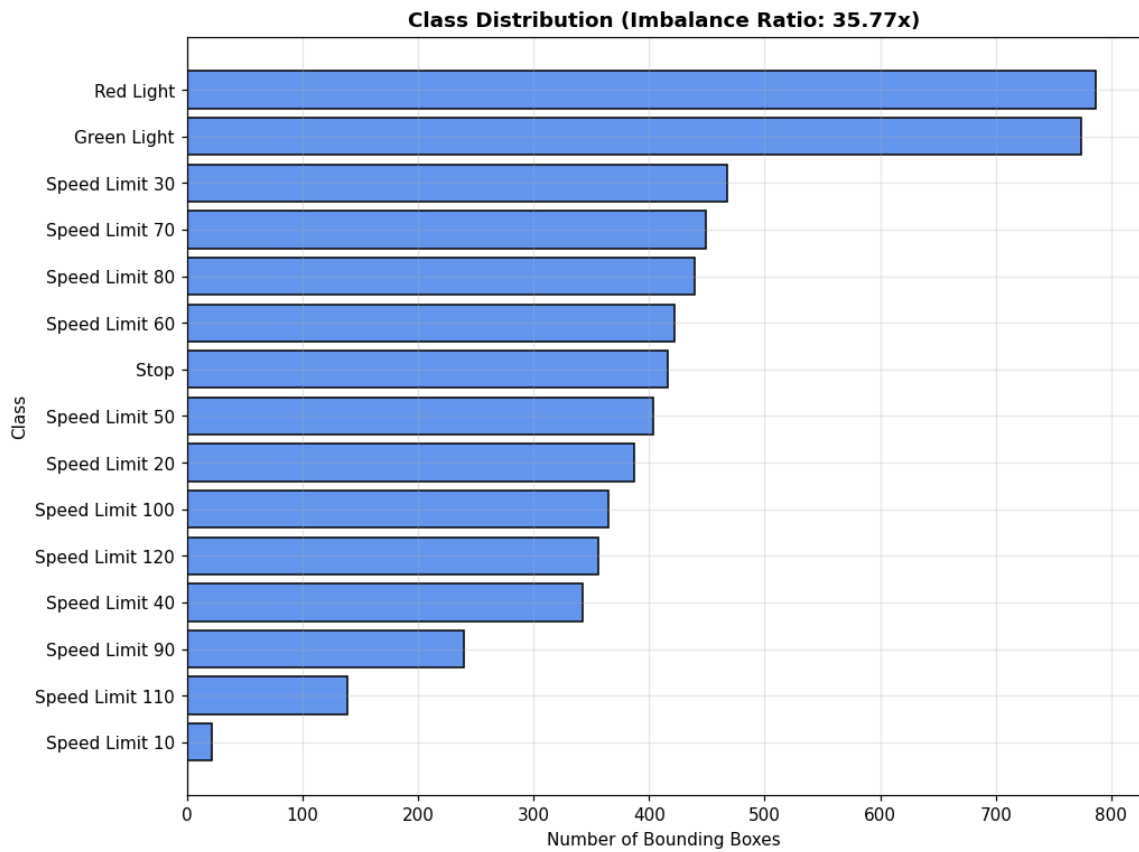
Bảng 3.1. Thống kê tổng quan bộ dữ liệu theo từng tập

Tập	Số ảnh	Tỉ lệ	Số hộp bao	Hộp/ảnh
Huấn luyện	3.530	71,0%	4.298	1,218
Kiểm định	801	16,1%	944	1,179
Kiểm tra	638	12,8%	770	1,207
Tổng	4.969	100%	6.012	1,210

Toàn bộ ảnh có độ phân giải đồng nhất 416×416 điểm ảnh — hệ quả của bước tiền xử lý mà Roboflow áp dụng khi xuất bộ dữ liệu. Chi tiết này tưởng nhỏ nhưng có hai hệ quả kỹ thuật quan trọng sẽ được bàn ở Mục 3.1.4: nó khiến việc tăng kích thước ảnh đầu vào khi huấn luyện không bổ sung thêm chi tiết thật, và nó khiến mọi ảnh trong một lô có cùng kích thước nên bước đệm trong nhánh DETR không bao giờ được kích hoạt.

Mật độ vật thể rất thưa: trung bình chỉ 1,21 hộp bao trên mỗi ảnh, cao nhất là 10 hộp ở tập kiểm tra. Đây là đặc điểm khác hẳn các bộ dữ liệu cảnh đường phố như TT100K [25], nơi một khung hình có thể chứa hàng chục biển báo.

3.1.2 Phân bố lớp và mất cân bằng



Hình 3.1. Phân bố số hộp bao theo từng lớp trên toàn bộ dữ liệu

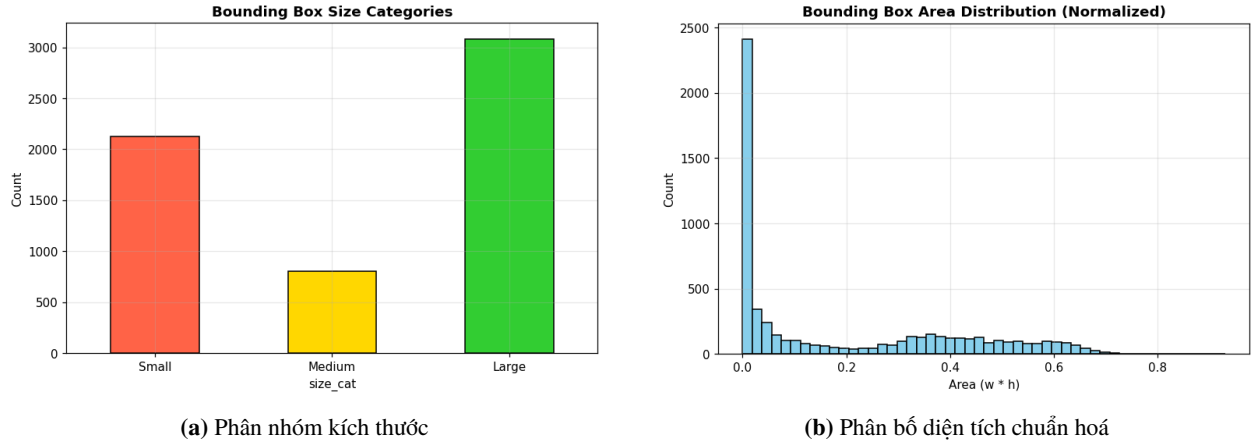
Hình 3.1 và Bảng 3.2 cho thấy mức mất cân bằng đáng kể. Hai lớp đèn tín hiệu chiếm gần 26% tổng số hộp bao, trong khi lớp *Speed Limit 10* chỉ có 22 mẫu, tương đương 0,37%. Tỷ số mất cân bằng, định nghĩa bằng thương giữa số mẫu của lớp nhiều nhất và lớp ít nhất, đạt **35,77**.

Bảng 3.2. Phân bố lớp chi tiết, sắp theo số hộp bao giảm dần

Mã lớp	Tên lớp	Số hộp	Tỉ lệ (%)	Tỉ số so với lớp lớn nhất
1	Red Light	787	13,09	1,00
0	Green Light	774	12,87	1,02
7	Speed Limit 30	468	7,78	1,68
11	Speed Limit 70	449	7,47	1,75
12	Speed Limit 80	440	7,32	1,79
10	Speed Limit 60	422	7,02	1,86
14	Stop	416	6,92	1,89
9	Speed Limit 50	404	6,72	1,95
6	Speed Limit 20	387	6,44	2,03
3	Speed Limit 100	365	6,07	2,16
5	Speed Limit 120	356	5,92	2,21
8	Speed Limit 40	343	5,71	2,29
13	Speed Limit 90	240	3,99	3,28
4	Speed Limit 110	139	2,31	5,66
2	Speed Limit 10	22	0,37	35,77

Khi cài đặt phép tính này, một chi tiết dễ sai đã được xử lý tường minh: nếu chỉ đếm các lớp thực sự xuất hiện trong dữ liệu, một lớp được khai báo trong tệp cấu hình nhưng không có mẫu nào sẽ bị âm thầm bỏ qua, khiến tỉ số mất cân bằng bị báo cáo thấp hơn thực tế và “lớp hiếm nhất” bị xác định sai. Mô-đun phân tích do đó đánh chỉ mục lại bảng đếm theo toàn bộ tập lớp khai báo với giá trị mặc định bằng không, và trả về vô cùng khi có lớp rỗng.

3.1.3 Thống kê kích thước hộp bao



Hình 3.2. Thống kê kích thước hộp bao trên toàn bộ dữ liệu

Việc phân nhóm kích thước dùng diện tích đã chuẩn hoá $a = w \cdot h$ với $w, h \in [0, 1]$, theo ba ngưỡng: nhỏ khi $a < 0,01$, trung bình khi $0,01 \leq a < 0,05$, và lớn trong các trường hợp còn lại. Cần lưu ý rằng quy ước này khác với quy ước COCO [7], vốn dùng diện tích tuyệt đối theo điểm ảnh với các ngưỡng 32^2 và 96^2 . Lựa chọn dùng diện tích chuẩn hoá là có chủ ý: nó độc lập với độ phân giải ảnh nên so sánh được giữa các nguồn dữ liệu khác nhau.

Bảng 3.3. Phân nhóm kích thước hộp bao theo diện tích chuẩn hoá

Nhóm	Số hộp	Tỉ lệ (%)	Ngưỡng diện tích chuẩn hoá
Nhỏ	2.122	35,30	$a < 0,01$
Trung bình	806	13,41	$0,01 \leq a < 0,05$
Lớn	3.084	51,30	$a \geq 0,05$

Bảng 3.3 chứa một tín hiệu bất thường. Trong một bộ dữ liệu cảnh đường phố thông thường, nhóm vật thể lớn hiếm khi chiếm quá bán, bởi biển báo chiếm hơn 5% diện tích khung hình đồng nghĩa với việc xe đã ở rất gần biển. Ở đây nhóm lớn chiếm tới 51,30%. Điều này thúc đẩy phân tích ở mục tiếp theo.

3.1.4 Phân tích thành phần bộ dữ liệu

Quan sát tên tệp cho thấy bộ dữ liệu không đồng nhất mà là một tập *trộn* từ nhiều nguồn có quy ước đặt tên khác nhau. Nhóm thực hiện phân loại ảnh theo mẫu tên tệp và tính lại thống

kê kích thước riêng cho từng nhóm; kết quả trình bày ở Bảng 3.4.

Bảng 3.4. Thành phần bộ dữ liệu theo nguồn gốc suy ra từ quy ước đặt tên tệp

Nhóm nguồn	Số ảnh	Tỉ lệ (%)	Cạnh biển (trung vị)	% vật thể nhỏ
Ảnh cắt kiểu GTSRB	1.922	38,68	66,8% cạnh ảnh	0,0
Ảnh số sáu chữ số	1.586	31,92	19,6%	34,7
Ảnh cảnh đường	724	14,57	13,4%	31,1
Camera mắt cá	645	12,98	1,6%	99,6
Nhóm khác	92	1,85	75,7%	0,0

Kết quả này giải thích trọn vẹn hình dạng bất thường ở Bảng 3.3. Nhóm ảnh cắt kiểu GTSRB chiếm 38,68% số ảnh và có biển báo lấp gần kín khung hình — đây vốn là ảnh dành cho bài toán *phân lớp* chứ không phải *phát hiện* [10]. Ở đầu kia của thang đo, nhóm camera mắt cá có tới 99,6% vật thể thuộc nhóm nhỏ, với cạnh biển trung vị chỉ 1,6% cạnh ảnh, đồng thời mang hình học quang học hoàn toàn khác.

Hệ quả là phân bố kích thước bị *lưỡng cực*. Bảng 3.5 trình bày các phân vị diện tích quy đổi ra cạnh tính bằng điểm ảnh trên ảnh 416×416 .

Bảng 3.5. Phân vị diện tích hộp bao và cạnh tương ứng trên ảnh 416×416

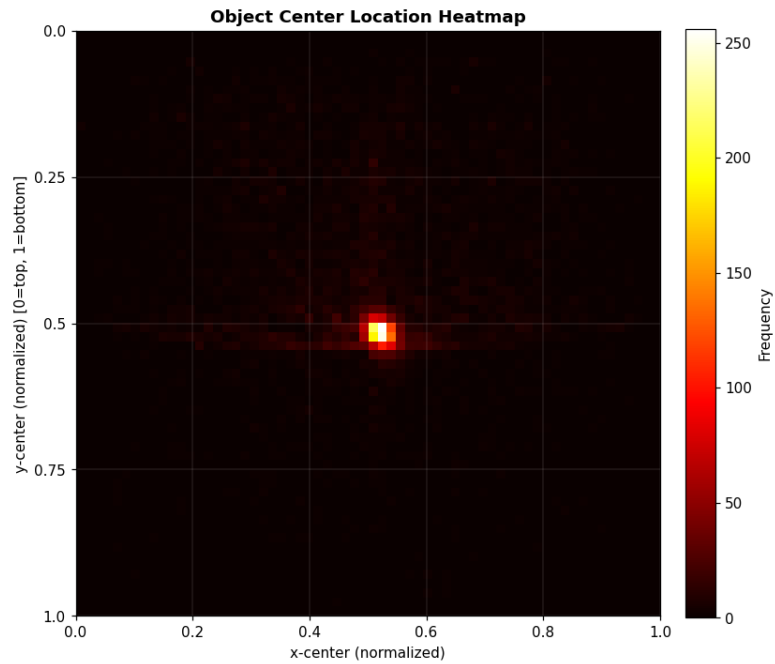
Phân vị	10%	25%	50%	75%	90%
Diện tích chuẩn hoá	0,0003	0,0025	0,0570	0,3854	0,5402
Cạnh tương ứng (điểm ảnh)	6,9	20,8	99,3	258,2	305,8

Khoảng cách giữa phân vị 25% (20,8 điểm ảnh) và phân vị 75% (258,2 điểm ảnh) là hơn mười hai lần, và vùng “biển báo thật trên đường” — khoảng 30 đến 80 điểm ảnh — gần như trống. Nói cách khác, mô hình được dạy chủ yếu bằng hai chế độ cực đoan mà thiếu hẳn chế độ trung gian vốn là chế độ thường gặp nhất trong thực tế. Đây là một dạng thiên lệch bộ dữ liệu điển hình [11] và là chìa khoá để diễn giải kết quả ở Chương 4.



Hình 3.3. Ba chế độ ảnh cùng tồn tại trong bộ dữ liệu, kèm hộp bao nhãn thật

3.1.5 Vị trí vật thể trong khung hình



Hình 3.4. Bản đồ nhiệt vị trí tâm vật thể trên toàn bộ dữ liệu

Bản đồ nhiệt ở Hình 3.4 được xây bằng biểu đồ tần suất hai chiều trên tọa độ tâm đã chuẩn hoá. Hai chi tiết cài đặt cần lưu ý. Thứ nhất, trục tung được lật theo phép $y' = 1 - y_c$ vì hệ tọa độ ảnh đặt gốc ở góc trên trái với trục y hướng xuống, trong khi hệ tọa độ đồ thị hướng lên. Thứ hai, ma trận tần suất phải được chuyển vị trước khi hiển thị, do hàm dựng biểu đồ hai

chiều trả về theo thứ tự (trục hoành, trục tung) còn hàm hiển thị ảnh yêu cầu thứ tự (hàng, cột). Bỏ qua một trong hai bước này sẽ cho một bản đồ nhiệt trông hợp lý nhưng sai hoàn toàn về mặt hình học.

3.1.6 Kiểm tra chất lượng nhãn

Một mô-đun kiểm tra độc lập rà soát tám loại lỗi: thiếu ảnh, thiếu nhãn, nhãn rỗng, ảnh hỏng, trùng tên ảnh giữa các tập, mã lớp ngoài miền hợp lệ, toạ độ hộp nằm ngoài đoạn $[0, 1]$ và hộp có cạnh bằng không. Kết quả trên bộ dữ liệu thật ghi nhận **4 ảnh có tập nhãn rỗng**, tức ảnh nền không chứa biển báo nào. Đây không phải lỗi mà là ảnh nền hợp lệ, thường được giữ lại có chủ đích để giảm dương tính giả.

Về mặt cài đặt, việc đối chiếu tên ảnh với tên tệp nhãn dùng cấu trúc tập hợp thay vì danh sách, nhờ đó phép kiểm tra thành viên có độ phức tạp hằng số thay vì tuyến tính, tránh chi phí bậc hai khi quy mô dữ liệu tăng.

3.2 Kiến trúc quy trình

Toàn bộ mã nguồn được tổ chức thành các gói độc lập, mỗi gói chịu trách nhiệm một giai đoạn của quy trình, và sổ tay Jupyter chỉ đóng vai trò điều phối. Bảng 3.6 tóm tắt.

Bảng 3.6. Tổ chức mô-đun của quy trình

Gói	Chức năng
<code>src/utils</code>	Đăng ký đường dẫn tập trung, tiện ích vẽ đồ thị, gieo hạt bộ sinh ngẫu nhiên
<code>src/data</code>	Dò định dạng, kiểm tra chất lượng nhãn, trực quan hoá chú thích, chuyển YOLO sang COCO
<code>src/eda</code>	Phân bố lớp, thống kê hộp bao, thống kê ảnh, bản đồ nhiệt vị trí
<code>src/training</code>	Huấn luyện YOLO và DETR
<code>src/evaluation</code>	Đánh giá độ chính xác, đo tốc độ, tổng hợp bảng so sánh
<code>tests</code>	43 trường hợp kiểm thử chạy trên CPU trong khoảng một giây

3.2.1 Bảo đảm tính tái lập

Ba biện pháp được áp dụng. Thứ nhất, một hàm gieo hạt duy nhất đặt hạt cho biến môi trường băm của Python, thư viện ngẫu nhiên chuẩn, NumPy, PyTorch và PyTorch CUDA, đồng thời bật chế độ tắt định của cuDNN và *tắt* chế độ tự dò thuật toán nhanh nhất. Việc tắt chế độ tự dò là cần thiết vì nó chọn thuật toán tích chập theo điều kiện thực thi tại thời điểm chạy, khiến kết quả không lặp lại được giữa các lần. Hạt mặc định là 42 và được truyền

tường minh vào cả bộ huấn luyện của Ultralytics.

Thứ hai, mọi đường dẫn đầu vào và đầu ra được khai báo tại một mô-đun duy nhất, có hỗ trợ biến môi trường để cùng một mã nguồn chạy được trên máy cá nhân lẫn trên Colab với dữ liệu đặt ở Google Drive. Riêng tệp cấu hình dữ liệu của Ultralytics được sinh lại với đường dẫn tuyệt đối tại thời điểm chạy, do thư viện này giải đường dẫn tương đối theo thư mục dữ liệu nội bộ của nó chứ không theo thư mục dự án.

Thứ ba, các thư viện học sâu được nhập *trễ*, tức chỉ nhập bên trong hàm cần dùng. Nhờ vậy các mô-đun phân tích dữ liệu và bộ kiểm thử chạy được trên máy chỉ có CPU, không cần cài đặt PyTorch [28] hay Ultralytics.

3.3 Thiết kế thực nghiệm

Bốn nhóm thực nghiệm được ký hiệu từ E0 đến E3, trình bày ở Bảng 3.7.

Bảng 3.7. Thiết kế bốn nhóm thực nghiệm

Mã	Câu hỏi	Cấu hình khảo sát
E0	Mốc so sánh là bao nhiêu?	YOLOv8n và DETR-ResNet50
E1	Có kiến trúc nhẹ nào không kém hơn?	YOLO11n, SSDLite-MobileNetV3, D-FINE-Nano
E2	Nén được mô hình tốt nhất mà không huấn luyện lại không?	FP16, ONNX INT8, OpenVINO INT8
E3	Học trò nhỏ có học được từ thầy lớn không?	YOLO26s dạy YOLO26n

3.3.1 Cấu hình huấn luyện nhánh YOLO

Mô hình nền là YOLOv8n, biến thể nhỏ nhất trong họ YOLOv8, khởi tạo từ trọng số tiền huấn luyện trên COCO với 80 lớp; đầu phân loại được thay để khớp 15 lớp của đề tài. Bảng 3.8 liệt kê các siêu tham số chính, trích trực tiếp từ tệp tham số mà Ultralytics ghi lại sau khi huấn luyện.

Bảng 3.8. Siêu tham số huấn luyện YOLOv8n

Tham số	Giá trị	Ghi chú
Số chu kỳ	30	Ngân sách cố định cho mọi cấu hình nhánh YOLO
Kích thước ảnh	640	Cân bằng giữa tốc độ và độ chính xác
Kích thước lô	16	
Hạt ngẫu nhiên	42	Truyền tường minh vào bộ huấn luyện
Tốc độ học đầu	0,01	Bộ tối ưu chọn tự động
Suy giảm trọng số	0,0005	
Chu kỳ khởi động	3	
Lật ngang	0,0	Tắt có chủ đích — xem giải thích bên dưới
Lật dọc	0,0	
Ghép ảnh mosaic	1,0	Tắt ở 10 chu kỳ cuối
Biến đổi tỉ lệ	0,5	
Tịnh tiến	0,1	
Nhiều màu HSV	0,015 / 0,7 / 0,4	Sắc độ / bão hoà / độ sáng
Xoá ngẫu nhiên	0,4	

Quyết định đáng chú ý nhất trong bảng trên là **tắt phép lật ngang**. Đây không phải giá trị mặc định của thư viện mà là lựa chọn xuất phát từ đặc thù bài toán: nhiều biển báo giao thông mang tính định hướng, chẳng hạn biển rẽ trái và biển rẽ phải chỉ khác nhau ở chiều mũi tên. Lật ngang một ảnh chứa biển rẽ trái sẽ tạo ra hình ảnh của biển rẽ phải nhưng vẫn giữ nhãn cũ, tức là sinh ra mẫu huấn luyện *sai*. Đây là ví dụ cho nguyên tắc rằng tăng cường dữ liệu phải phù hợp với ngữ nghĩa của miền ứng dụng chứ không thể áp dụng máy móc [29].

Ngược lại, phép ghép ảnh mosaic được giữ ở hệ số tối đa. Kỹ thuật này, phổ biến từ YOLOv4 [30], ghép bốn ảnh huấn luyện thành một, nhờ đó vừa tăng số ngữ cảnh nền mà mỗi vật thể xuất hiện, vừa tạo ra các vật thể ở tỉ lệ nhỏ hơn — đúng chế độ mà bộ dữ liệu đang thiếu. Mosaic được tắt ở 10 chu kỳ cuối để mô hình kết thúc quá trình huấn luyện trên phân bố ảnh gần với phân bố lúc suy luận.

3.3.2 Cấu hình huấn luyện nhánh DETR

Nhánh DETR tinh chỉnh mô hình facebook/detr-resnet-50 qua thư viện Transformers [31]. Vì trọng số gốc có đầu phân loại 91 lớp trong khi đề tài cần 15 lớp, cờ bỏ qua sai lệch kích

thước được bật để giữ lại phần xương sống và transformer, chỉ khởi tạo lại đầu phân loại. Nhật ký huấn luyện xác nhận đầu này được tạo mới với kích thước 16×256 , tức 15 lớp cộng một lớp “không có vật thể” do mô hình tự quản lý.

Bảng 3.9. Siêu tham số huấn luyện DETR-ResNet50

Tham số	Giá trị	Ghi chú
Số chu kỳ	10	Giới hạn bởi ngân sách GPU
Kích thước lô	2	Giới hạn bởi bộ nhớ card đồ hoạ
Tốc độ học đầu	$1,0 \times 10^{-5}$	Thấp hơn công thức gốc một bậc
Tốc độ học xương sống	$1,0 \times 10^{-6}$	
Bộ tối ưu	AdamW	Suy giảm trọng số 10^{-4} [32]
Cắt chuẩn gradient	0,1	Chống bùng nổ gradient [33]
Số lô huấn luyện	883	
Số lô kiểm định	201	

Ba biện pháp ổn định số học được áp dụng sau khi công thức gốc của thư viện, vốn dùng tốc độ học 10^{-4} cho phần đầu, làm hàm mất mát phân kỳ về giá trị không xác định. Thứ nhất, tốc độ học được hạ một bậc cho cả hai nhóm tham số. Thứ hai, mỗi bước huấn luyện kiểm tra tính hữu hạn của hàm mất mát và bỏ qua lô hiện tại nếu phát hiện giá trị không hợp lệ, tránh việc một bước hỏng làm nhiễm toàn bộ trọng số. Thứ ba, chuẩn gradient được cắt ở ngưỡng 0,1.

Việc tách tốc độ học giữa xương sống và phần đầu là một kỹ thuật tinh chỉnh phân biệt: xương sống ResNet-50 [34] đã được tiền huấn luyện tốt nên chỉ cần điều chỉnh nhẹ, trong khi phần đầu vừa được khởi tạo ngẫu nhiên nên cần học nhanh hơn. Mã nguồn phân nhóm tham số theo tên rồi truyền hai nhóm tốc độ học riêng vào bộ tối ưu.

3.3.3 Cấu hình nhóm E3 — chứng cất tri thức

Nhóm E3 được tổ chức chặt chẽ hơn ba nhóm còn lại và đáng được mô tả riêng, vì nó là nhóm duy nhất có đối chứng đúng nghĩa và có tiêu chí chấp nhận khai báo trước.

Bảng 3.10. Siêu tham số nhóm E3

Tham số	Giá trị	Ghi chú
Mô hình thầy	yo1o26s.pt	9,47 triệu tham số
Mô hình trò	yo1o26n.pt	2,38 triệu tham số
Số chu kỳ	50 / 50 / 50	Thầy, trò đối chứng, trò chưng cất — bằng nhau
Kích thước lô	16 / 24 / 8	Không bằng nhau — xem ghi chú bên dưới
Kiên nhẫn dừng sớm	12	
Hệ số chưng cất	6,0	Giá trị mặc định của thư viện
Kích thước ảnh	640	Giống nhánh YOLO ở nhóm E0
Hạt ngẫu nhiên	42	Giống toàn bộ đề tài
Ngưỡng tin cậy khi đo	0,001	Gần bằng không, phù hợp với độ đo xếp hạng
Số ảnh hiệu chuẩn INT8	300	
Số lượt khởi động / đo tốc độ	10 / 100	

Cần nêu rõ một **yếu tố gây nhiễu chưa kiểm soát được**: nhánh chưng cất chạy với kích thước lô bằng 8, trong khi nhánh đối chứng dùng 24. Nguyên nhân là trong quá trình chưng cất phải nạp đồng thời cả mô hình thầy lẫn mô hình trò vào bộ nhớ card đồ họa. Vì kích thước lô ảnh hưởng tới chất lượng ước lượng gradient, phần chênh lệch quan sát được ở Chương 4 không thể quy hoàn toàn cho bản thân kỹ thuật chưng cất. Đây là hạn chế được ghi nhận thay vì bỏ qua.

Nhóm E3 cũng khác ba nhóm còn lại ở khâu đánh giá: thay vì dùng bộ đánh giá của thư viện huấn luyện, nó chuyển nhãn sang định dạng COCO rồi dùng **pycocotools**. Nhờ vậy có thêm *AP* tách theo ba nhóm kích thước — chính là chỉ số cần thiết để đọc đúng ảnh hưởng của các phép nén mô hình lên vật thể nhỏ.

Cuối cùng, năm ngưỡng chấp nhận được khai báo *trước* khi chạy thực nghiệm, chẳng hạn “mô hình chưng cất không được kém mô hình đối chứng quá 0,5 điểm $mAP@0,5:0,95$ ” hay “INT8 phải nhỏ hơn FP32 ít nhất 50%”. Cách làm này buộc người thực hiện phải cam kết tiêu chí đánh giá trước khi nhìn thấy số liệu, tránh việc điều chỉnh kỳ vọng cho khớp với kết quả thu được.

3.3.4 Kiểm toán rò rỉ dữ liệu giữa các tập

Một thủ tục kiểm toán độc lập được thực hiện để phát hiện ảnh xuất hiện ở nhiều hơn một tập. Thủ tục đối chiếu theo hai mức: tên ảnh nguồn sau khi loại bỏ hậu tố ngẫu nhiên do

Roboflow thêm vào, và băm nội dung tệp để xác định các cặp trùng byte hoàn toàn. Kết quả được trình bày ở Mục 4.5 và ảnh hưởng trực tiếp tới cách diễn giải mọi chỉ số trong báo cáo.

3.3.5 Quy trình đánh giá

Mọi mô hình được đánh giá trên tập kiểm tra tách biệt, không tham gia vào bất kỳ quyết định nào trong quá trình huấn luyện. Điểm kiểm tra được chọn theo chỉ số trên tập kiểm định — Ultralytics chọn theo điểm thích nghi tổng hợp, còn nhánh DETR lưu điểm kiểm tra có mất mát kiểm định thấp nhất thay vì chu kỳ cuối, để trọng số trên đĩa khớp với chu kỳ tốt nhất được báo cáo.

Khi tính mAP cho nhánh DETR, ngưỡng tin cậy được đặt bằng **không** và toàn bộ 100 truy vấn được giữ lại. Đây là điểm dễ gây hiểu nhầm nên cần nói rõ: mAP là độ đo xếp hạng, cần trọn bộ dự đoán có điểm số để dựng đầy đủ đường cong PR. Lọc theo ngưỡng tin cậy trước khi tính sẽ cắt mất phần đuôi độ bao phủ và làm sai lệch chỉ số. Ngưỡng tin cậy chỉ được dùng ở khâu trình diễn.

Tốc độ suy luận được đo bằng cách chạy nhiều lượt truyền xuôi trên đầu vào giả sau một giai đoạn khởi động, rồi lấy trung bình. Giai đoạn khởi động là bắt buộc vì lượt chạy đầu tiên chịu chi phí khởi tạo ngữ cảnh CUDA và dò thuật toán, sẽ làm phồng số đo nếu không loại bỏ.

CHƯƠNG 4 : KẾT QUẢ VÀ BÀN LUẬN

4.1 Kết quả tổng quan

4.1.1 Hai giai đoạn thực nghiệm

Đề tài được tiến hành qua **hai giai đoạn** có mục tiêu khác nhau, và điều này quyết định cách đọc các bảng số liệu phía sau.

Giai đoạn một tập trung vào phân tích dữ liệu và thiết lập mốc so sánh. Câu hỏi đặt ra là hai họ kiến trúc lớn — tích chập một giai đoạn và transformer dự đoán theo tập hợp — khác nhau ra sao trên bộ dữ liệu này. Đây là nội dung của nhóm E0, với YOLOv8n đối đầu DETR-ResNet50. Kết quả được ghi ra tệp chỉ số bởi mô-đun đánh giá, lưu kèm đường dẫn trọng số, tên tập đánh giá và thiết bị đo, nên truy vết được đầy đủ.

Giai đoạn hai chuyển trọng tâm sang bài toán triển khai. Sau khi giai đoạn một đã cho thấy DETR không hội tụ kịp trong ngân sách hiện có, việc tiếp tục so với DETR không còn giá trị thông tin; câu hỏi mới trở thành làm sao đưa mô hình tốt nhất xuống phần cứng hạn chế. Đó là lý do các nhóm E1, E2 và E3 so sánh với những đối tượng khác — kiến trúc nhẹ, mô hình lượng tử hoá, mô hình chưng cất — thay vì với DETR.

Vì hai giai đoạn cách nhau về thời gian và chạy trên hai cấu hình phần cứng khác nhau, *các con số giữa hai giai đoạn không so sánh trực tiếp được với nhau*. Cấu hình YOLOv8n xuất hiện ở cả hai và cho hai bộ số: 0,9539 với 113,2 khung hình mỗi giây ở giai đoạn một, so với 0,9633 và 65,3 ở giai đoạn hai. Báo cáo *không* chọn một con số rồi bỏ con số kia, mà ghi nhận cả hai kèm nhãn giai đoạn, và mọi phép so sánh chỉ được thực hiện *trong nội bộ từng giai đoạn*.

4.1.2 Bảng tổng hợp

Bảng 4.1. Tổng hợp kết quả bốn nhóm thực nghiệm trên tập kiểm tra

Mã	Cấu hình	Cách làm	mAP@0,5	mAP@0,5:0,95	FPS	GD
<i>Giai đoạn 1 — mốc so sánh giữa hai họ kiến trúc</i>						
E0	YOLOv8n	Mô hình nền, 30 chu kỳ	0,9539	0,8058	113,2	1
E0	DETR-ResNet50	Transformer nền, 10 chu kỳ	0,1220	0,0988	15,7	1
<i>Giai đoạn 2 — bài toán triển khai, phần cứng khác</i>						
–	YOLOv8n FP32	<i>Mốc tham chiếu giai đoạn 2</i>	0,9633	0,8068	65,3	2
E1	YOLO11n	Thế hệ YOLO mới hơn, nhẹ hơn	0,9252	0,7833	59,0	2
E1	SSDLite-MobileNetV3	CNN di động, 320 điểm ảnh	0,7432	0,6260	57,9	2
E1	D-FINE-Nano	Transformer thế hệ mới, nhẹ	0,7836	0,6589	25,8	2
E2	PyTorch FP16	Giữ YOLOv8n, đổi sang FP16	0,9633	0,8068	65,3	2
E2	ONNX INT8	Lượng tử hoá sau huấn luyện	0,9552	0,7973	5,7	2
E2	OpenVINO INT8	INT8 qua OpenVINO trên CPU	0,9265	0,7846	13,9	2
E3	YOLO26n đối chứng	Học trò huấn luyện thường, 50 chu kỳ	0,9348	0,7773	69,7	2
E3	YOLO26n chưng cất	Học từ YOLO26s, hệ số 6,0	0,9014	0,7450	74,8	2

Nhận xét. Trong nội bộ giai đoạn 2, không một cấu hình nào trong tám cấu hình khảo sát vượt được mốc tham chiếu YOLOv8n FP32. Kết quả này đi ngược giả thuyết ban đầu ở cả nhóm E1 lẫn nhóm E3, và được báo cáo đầy đủ ở Mục 4.5.

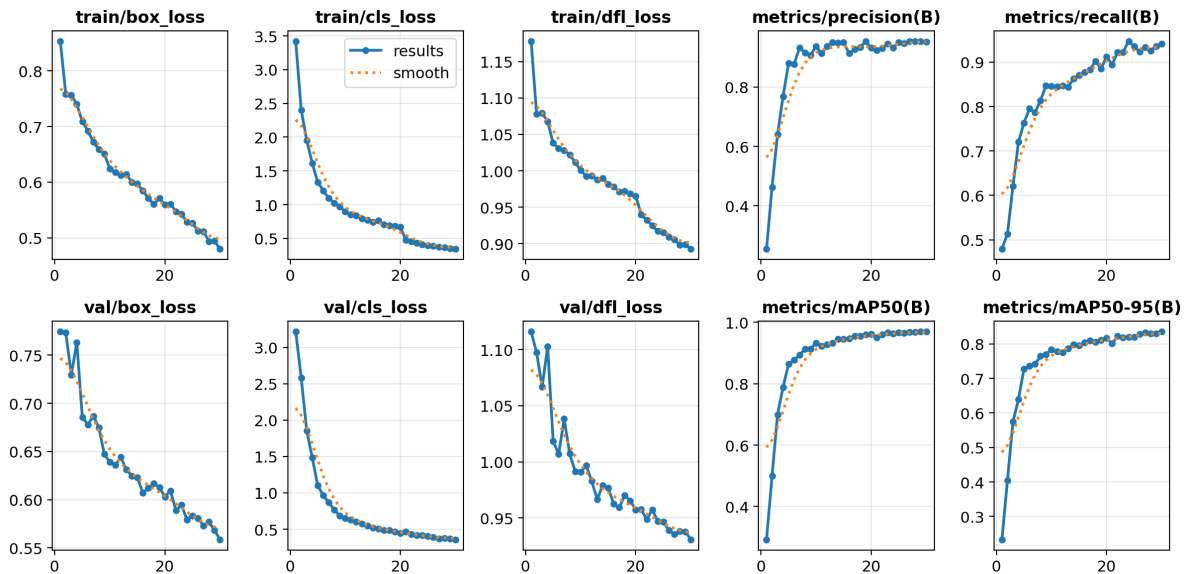
4.2 E0 — So sánh mô hình nền

4.2.1 Kết quả chi tiết

Bảng 4.2. So sánh chi tiết YOLOv8n và DETR-ResNet50 trên tập kiểm tra

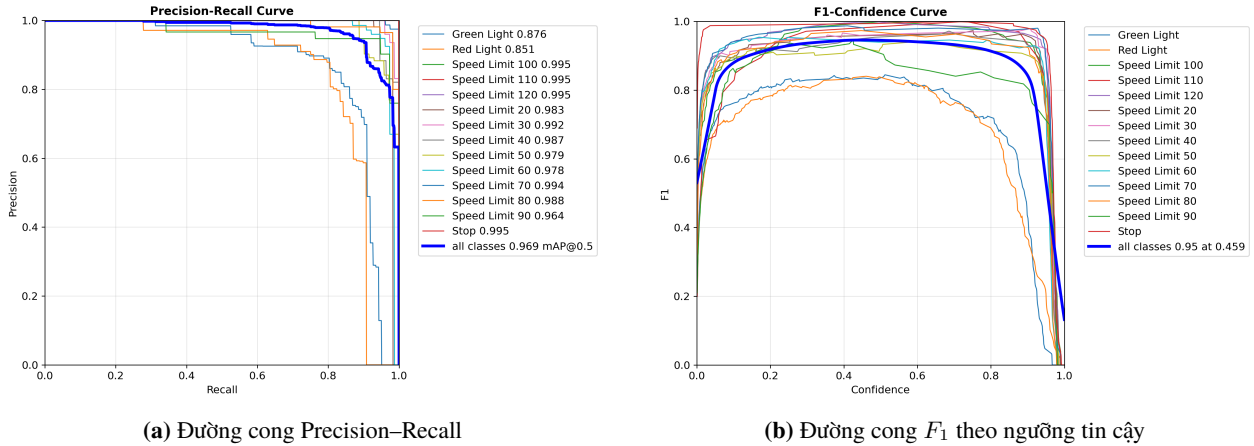
Chỉ số	YOLOv8n	DETR-ResNet50
mAP@0,5	0,9539	0,1220
mAP@0,5:0,95	0,8058	0,0988
Precision	0,9084	—
Recall	0,9564	—
Độ trễ (ms/ảnh)	8,83	63,83
Tốc độ (FPS)	113,2	15,7
Dung lượng (MB)	6,25	166,02
Số chu kỳ	30	10
Kích thước lô	16	2

Toàn bộ Bảng 4.2 thuộc giai đoạn 1 theo Mục 4.1.1, nên hai cột so sánh được trực tiếp với nhau: cùng tập kiểm tra, cùng phần cứng, cùng mô-đun đánh giá. Riêng Precision và Recall chỉ có ở nhánh YOLO vì mô-đun đánh giá DETR không ghi hai chỉ số này ra tập kết quả — một thiếu sót của quy trình được nêu ở phần hạn chế.



Hình 4.1. Diễn biến hàm mất mát và các chỉ số của YOLOv8n qua 30 chu kỳ huấn luyện

Hình 4.1 cho thấy quá trình huấn luyện diễn ra lành mạnh: cả ba thành phần mất mát đều giảm đơn điệu, mất mát trên tập kiểm định không tách rời khỏi mất mát huấn luyện, và các chỉ số bão hoà dần từ khoảng chu kỳ thứ 20. Chỉ số trên tập kiểm định ở chu kỳ cuối đạt $mAP@0,5$ bằng 0,9695 và $mAP@0,5:0,95$ bằng 0,8354. Khoảng cách giữa con số này và con số trên tập kiểm tra (0,9539 và 0,8058) là nhỏ và theo đúng chiều mong đợi, cho thấy mô hình không bị quá khớp nghiêm trọng.



Hình 4.2. Đặc tính phân loại của YOLOv8n trên tập kiểm định

4.2.2 Vì sao DETR kém hơn tới gần tám lần

Chênh lệch giữa 0,9539 và 0,1220 lớn tới mức không thể quy cho một nguyên nhân đơn lẻ. Phân tích cho thấy có ít nhất năm yếu tố cùng tác động, và *không yếu tố nào trong số đó là bằng chứng cho thấy kiến trúc transformer kém hơn về bản chất*.

1. **Ngân sách huấn luyện chênh lệch ba lần.** YOLO được huấn luyện 30 chu kỳ, DETR chỉ 10.
2. **Tốc độ hội tụ đặc thù của DETR.** Bản gốc cần khoảng 300 đến 500 chu kỳ trên COCO [4]. Với 10 chu kỳ, mô hình về cơ bản vẫn đang ở giai đoạn đầu của quá trình học phép gán Hungarian.
3. **Kích thước lô quá nhỏ.** Lô 2 ảnh, bị giới hạn bởi bộ nhớ card đồ hoạ, khiến ước lượng gradient nhiễu và phép ghép cặp song ánh dao động mạnh giữa các bước.
4. **Tốc độ học bị hạ để đổi lấy ổn định.** Việc hạ một bậc so với công thức gốc là cần thiết để tránh phân kỳ, nhưng cái giá là học chậm hơn nhiều.

5. **Tăng cường dữ liệu hoàn toàn bất đối xứng.** Đây là yếu tố ít được nhắc tới nhất nhưng có thể là yếu tố nặng ký nhất. Nhánh YOLO nhận đầy đủ bộ tăng cường của Ultralytics gồm ghép ảnh mosaic hệ số 1,0, biến đổi tỉ lệ 0,5, tịnh tiến 0,1, nhiễu màu HSV và xoá ngẫu nhiên 0,4. Nhánh DETR *không có bất kỳ phép tăng cường nào* — bộ dữ liệu chỉ gọi bộ tiền xử lý ảnh rồi trả về. Với một kiến trúc vốn đã đói dữ liệu [18], việc bị tước toàn bộ tăng cường là bất lợi rất lớn.

Thêm vào đó, kiến trúc transformer thiếu thiên kiến quy nạp về tính địa phương và bất biến tịnh tiến vốn có sẵn trong mạng tích chập. Trên một bộ dữ liệu chỉ 6.012 hộp bao, sự thiếu vắng này trở thành bất lợi rõ rệt. Kết luận đúng đắn rút ra từ E0 vì vậy không phải “DETR kém hơn YOLO” mà là “ở ngân sách dữ liệu và tính toán của đề tài này, YOLO hiệu quả hơn hẳn, còn DETR mới chỉ ở trạng thái chưa huấn luyện đủ”. Các công trình gần đây như RT-DETR [20] cho thấy khi được huấn luyện đầy đủ, dòng transformer hoàn toàn có thể vượt YOLO.

4.3 E1 — Khảo sát kiến trúc nhẹ

Ba kiến trúc thay thế được thử với kỳ vọng tìm ra phương án nhẹ hơn mà không kém hơn. Cả ba đều không đạt kỳ vọng.

Mọi con số trong mục này thuộc giai đoạn 2, nên được so với mốc tham chiếu YOLOv8n FP32 của chính giai đoạn đó (0,9633) chứ không so với con số ở Bảng 4.2.

YOLO11n, thuộc thế hệ mới hơn YOLOv8n, đạt mAP@0,5 bằng 0,9252 — thấp hơn mốc tham chiếu 3,81 điểm phần trăm, đồng thời chậm hơn. Đây là kết quả tốt nhất trong nhóm E1 nhưng vẫn không đủ để thay thế.

SSDLite-MobileNetV3 [3, 35] cho kết quả thấp nhất nhóm với 0,7432. Đáng chú ý hơn là khoảng cách giữa hai độ đo: mAP@0,5:0,95 chỉ đạt 0,6260, tức mô hình định vị hộp bao kém khít. Điều này phù hợp với đặc điểm kiến trúc: SSDLite chạy ở độ phân giải đầu vào chỉ 320 điểm ảnh và dùng xương sống rất nhẹ, nên gần như mù với vật thể nhỏ — vốn chiếm 35,30% số hộp bao trong bộ dữ liệu này.

D-FINE-Nano đạt 0,7836, tốt hơn SSDLite về định vị nhưng yếu hơn về phân loại. Điểm đáng ghi nhận là nó cho thấy một kiến trúc transformer *được thiết kế để hội tụ nhanh* có thể đạt mức chấp nhận được trong cùng ngân sách mà DETR gốc chỉ đạt 0,1220 — một xác nhận gián tiếp cho lập luận ở Mục 4.2.2 rằng vấn đề của DETR nằm ở tốc độ hội tụ chứ không ở kiến trúc transformer nói chung.

4.4 E2 — Lượng tử hoá sau huấn luyện

Bảng 4.3. Ảnh hưởng của lượng tử hoá tới độ chính xác và tốc độ (toàn bộ thuộc giai đoạn 2)

Cấu hình	mAP@0,5	Δ so với gốc	FPS	Nhận xét
PyTorch FP32 (gốc)	0,9633	—	65,3	Mốc so sánh giai đoạn 2
PyTorch FP16	0,9633	0,0000	65,3	Không mất độ chính xác
ONNX INT8	0,9552	-0,0081	5,7	Nén tốt nhất, chậm nhất
OpenVINO INT8	0,9265	-0,0368	13,9	Nhanh hơn ONNX INT8

FP16 giữ nguyên hoàn toàn độ chính xác trong khi giảm khoảng một nửa dung lượng — kết quả phù hợp với các nghiên cứu về huấn luyện độ chính xác hỗn hợp [22]. Đây là lựa chọn không có nhược điểm trong bối cảnh này.

Kết quả INT8 thì trái với kỳ vọng thông thường. Về lý thuyết, biểu diễn số nguyên 8 bit cho phép dùng các nhân tính toán số nguyên nhanh hơn [21]. Trên thực tế, ONNX INT8 chỉ đạt 5,7 khung hình mỗi giây, tức chậm hơn bản FP32 hơn mười một lần. Nguyên nhân nhiều khả năng là môi trường thực thi CPU dùng trong thí nghiệm không có nhân số nguyên được tối ưu cho mẫu tính toán của mô hình, nên phần lớn thời gian bị tiêu vào các phép chuyển đổi lượng tử hoá và giải lượng tử hoá xen giữa các lớp. OpenVINO INT8 nhanh hơn ONNX INT8 khoảng 2,4 lần nhưng đánh đổi 3,68 điểm phần trăm mAP.

Đáng chú ý hơn con số tổng: phần suy giảm do lượng tử hoá INT8 **tập trung vào nhóm vật thể nhỏ**. Nhóm E2 không đo *AP* tách theo kích thước nên không định lượng được trực tiếp, nhưng nhóm E3 — vốn có thí nghiệm lượng tử hoá riêng và được đo bằng pycocotools — cho con số cụ thể: khi chuyển từ ONNX FP32 sang ONNX INT8, mAP@0,5 chỉ giảm 0,0038 trong khi *AP* trên vật thể nhỏ giảm 0,0247, tức **gấp 6,5 lần** (xem Bảng 4.5).

Điều này hợp lý về mặt cơ chế: lượng tử hoá làm giảm độ phân giải biểu diễn của các giá trị kích hoạt, và vật thể nhỏ vốn đã cho tín hiệu yếu trên bản đồ đặc trưng nên chịu thiệt nhiều nhất. Hệ quả thực tiễn rất cụ thể: với bài toán biển báo giao thông, nơi 35,30% hộp bao thuộc nhóm nhỏ và biển ở xa mới là trường hợp quan trọng, một cấu hình INT8 nhìn qua “chỉ mất vài phần nghìn mAP” có thể mất nhiều hơn hẳn ở đúng nhóm cần quan tâm. Khi chọn cấu hình triển khai, phải đọc *AP* tách theo nhóm kích thước chứ không chỉ đọc mAP tổng.

Nhận xét. Bài học rút ra là lượng tử hoá INT8 không tự động đồng nghĩa với suy luận nhanh hơn; lợi ích của nó phụ thuộc chặt vào môi trường thực thi và phần cứng đích. Trong bối cảnh của đề tài, FP16 là lựa chọn hợp lý duy nhất.

4.5 Kết quả trái với kỳ vọng

4.5.1 E3 — Chứng cất tri thức cho kết quả âm

Giả thuyết ban đầu là mô hình học trò YOLO26n sẽ hưởng lợi từ phân phối đầu ra mềm của mô hình thầy YOLO26s [23]. Kết quả ngược lại.

Khác với các nhóm còn lại, nhóm E3 được đánh giá bằng **pycocotools** trên tập chú thích COCO của tập kiểm tra, nên ngoài mAP tổng còn có *AP* tách theo nhóm kích thước. Bảng 4.4 trình bày đầy đủ, bao gồm cả hai cấu hình xuất sang ONNX của chính mô hình chứng cất.

Bảng 4.4. Kết quả nhóm E3 đo bằng pycocotools trên tập kiểm tra

Cấu hình	Tham số (M)	Tệp (MB)	mAP@0,5	mAP@.5:.95	<i>AP</i> nhỏ	<i>AP</i> lớn	FPS
YOLO26s (thầy)	9,47	19,39	0,9520	0,7822	0,616	0,859	74,4
YOLO26n đối chứng	2,38	5,15	0,9348	0,7773	0,548	0,862	69,7
YOLO26n chứng cất	2,38	5,15	0,9014	0,7450	0,486	0,852	74,8
ONNX FP32 (từ trò KD)	—	9,35	0,9079	0,7473	0,462	0,847	19,8
ONNX INT8 (từ trò KD)	—	2,78	0,9041	0,7437	0,437	0,839	9,4

Chứng cất thực sự đã được kích hoạt

Trước khi diễn giải một kết quả âm, cần loại trừ khả năng cơ chế chứng cất không hề chạy. Nhật ký huấn luyện của nhánh chứng cất có thêm cột `train/dis_loss` — cột này không tồn tại ở nhánh đối chứng — và giá trị của nó giảm từ 12,1663 ở chu kỳ đầu xuống 1,1608 ở chu kỳ cuối. Hàm mất mát chứng cất đã hoạt động và đã hội tụ; kết quả âm là kết quả thật chứ không phải lỗi cấu hình.

Suy giảm tập trung ở vật thể nhỏ

Bảng 4.5. Ba phép so sánh cốt lõi của nhóm E3

Phép so sánh	$\Delta \text{mAP}@0,5$	$\Delta \text{mAP}@.5:.95$	ΔAP nhỏ
Trò chứng cất so với trò đối chứng	−0,0334	−0,0323	− 0,0620
Trò chứng cất so với thầy	−0,0507	−0,0372	− 0,1308
ONNX INT8 so với ONNX FP32	−0,0038	−0,0036	− 0,0247

Nhận xét. Cột AP nhỏ là cột đáng đọc nhất. Ở cả ba phép so sánh, mức suy giảm trên vật thể nhỏ đều lớn hơn mức suy giảm trên mAP tổng — gấp 1,9 lần ở phép chứng cất và gấp **6,5 lần** ở phép lượng tử hoá INT8. Một cấu hình nhìn qua chỉ “mất 0,38 điểm mAP ” thực chất mất 2,47 điểm ở đúng nhóm vật thể quan trọng nhất đối với bài toán biển báo giao thông.

Cột AP lớn cho thấy điều ngược lại: cả ba mô hình PyTorch đều nằm quanh mức 0,85–0,86, gần như không phân biệt được. Nói cách khác, mọi khác biệt giữa các cấu hình đều nằm ở vật thể nhỏ, còn với vật thể lớn thì bài toán đã bão hoà. Điều này nhất quán với phân tích thành phần bộ dữ liệu ở Mục 3.1.4: hơn một nửa số hộp bao thuộc nhóm lớn và chúng quá dễ để tạo ra khác biệt giữa các mô hình.

Vì sao chứng cất thất bại

Mô hình được chứng cất thấp hơn đối chứng 3,34 điểm phần trăm $\text{mAP}@0,5$ và 6,20 điểm phần trăm AP trên vật thể nhỏ. Có ba cách giải thích khả dĩ. Thứ nhất, khoảng cách năng lực giữa thầy và trò quá hẹp: thầy chỉ đạt 0,9520, tức nhỉnh hơn trò đối chứng 1,72 điểm phần trăm, nên lượng “tri thức tối” có thể truyền đạt là rất ít — trong khi chi phí nhiều do bắt chước một phân phối không hoàn hảo thì vẫn phải trả đủ. Thứ hai, kích thước lô của nhánh chứng cất chỉ bằng 8 so với 24 của nhánh đối chứng, do phải nạp đồng thời cả hai mô hình vào bộ nhớ card đồ hoạ; đây là một yếu tố gây nhiễu chưa được kiểm soát và tự nó đã đủ giải thích một phần chênh lệch. Thứ ba, hệ số chứng cất 6,0 — giá trị mặc định của thư viện — có thể quá lớn với một cặp thầy–trò gần nhau như vậy, khiến thành phần bắt chước lẫn át tín hiệu từ nhãn cứng.

Đáng chú ý, thí nghiệm được thiết kế kèm *ngưỡng chấp nhận khai báo trước khi chạy*.

Trong năm ngưỡng, ba đạt và hai không đạt — cả hai ngưỡng không đạt đều thuộc về giả thuyết chưng cất. Việc khai báo tiêu chí trước và báo cáo cả kết quả không đạt là cách làm đúng đắn về mặt phương pháp: nó ngăn việc diễn giải lại kỳ vọng sau khi đã nhìn thấy số liệu.

Bảng 4.6. Kiểm tra các ngưỡng chấp nhận khai báo trước khi chạy thực nghiệm

Ngưỡng	Giá trị đo	Kết quả
KD không kém đối chứng quá 0,5 điểm	$\Delta = -0,0323$	KHÔNG ĐẠT
KD tốt hơn đối chứng (kỳ vọng ban đầu)	$\Delta = -0,0323$	KHÔNG ĐẠT
INT8 giảm không quá 1,5 điểm so với FP32	giảm 0,0036	ĐẠT
INT8 nhỏ hơn FP32 ít nhất 50%	giảm 70,3%	ĐẠT
Trò chưng cất nhẹ hơn thầy	5,15 so với 19,39 MB	ĐẠT

Kết quả này được giữ lại trong báo cáo thay vì loại bỏ, vì nó xác định rõ *điều kiện áp dụng* của kỹ thuật: chưng cất tri thức chỉ có ý nghĩa khi tồn tại khoảng cách năng lực đủ lớn giữa thầy và trò, và khi mô hình thầy thực sự mạnh. Muốn kết luận chắc chắn hơn, cần chạy lại nhánh chưng cất với cùng kích thước lô như nhánh đối chứng và quét hệ số chưng cất thay vì dùng giá trị mặc định.

4.5.2 Khoảng cách giữa chỉ số và hành vi thực tế

Kết quả trái kỳ vọng thứ hai mang tính hệ thống hơn. Mô hình đạt mAP@0,5 bằng 0,9539 trên tập kiểm tra, nhưng khi chạy trên video đường phố thực tế thu bằng camera hành trình, khả năng phát hiện suy giảm rõ rệt, đặc biệt với biển báo ở xa.

Phân tích thành phần bộ dữ liệu ở Mục 3.1.4 giải thích hiện tượng này. Với 38,68% số ảnh huấn luyện là ảnh cắt cận cảnh có cạnh biển chiếm 66,8% cạnh ảnh, mô hình được dạy rất kỹ chế độ “biển lấp đầy khung hình”. Trong khi đó, chế độ thực sự quan trọng khi lái xe — biển báo cách 30 đến 50 mét, tương ứng khoảng 30 đến 80 điểm ảnh trong khung hình Full HD — gần như không xuất hiện trong dữ liệu, như Bảng 3.5 đã chỉ ra.

Vấn đề càng nghiêm trọng hơn ở khâu suy luận. Khi một khung hình 1920×1080 được đưa vào mô hình huấn luyện ở kích thước 640, thư viện sẽ co ảnh theo hệ số khoảng 0,33. Một biển báo 40 điểm ảnh trở thành 13 điểm ảnh, chỉ trải trên khoảng 1,6 ô lưới của đầu dự đoán P3 vốn hoạt động ở bước sải 8 — tức nằm ngay ở giới hạn khả năng phát hiện.

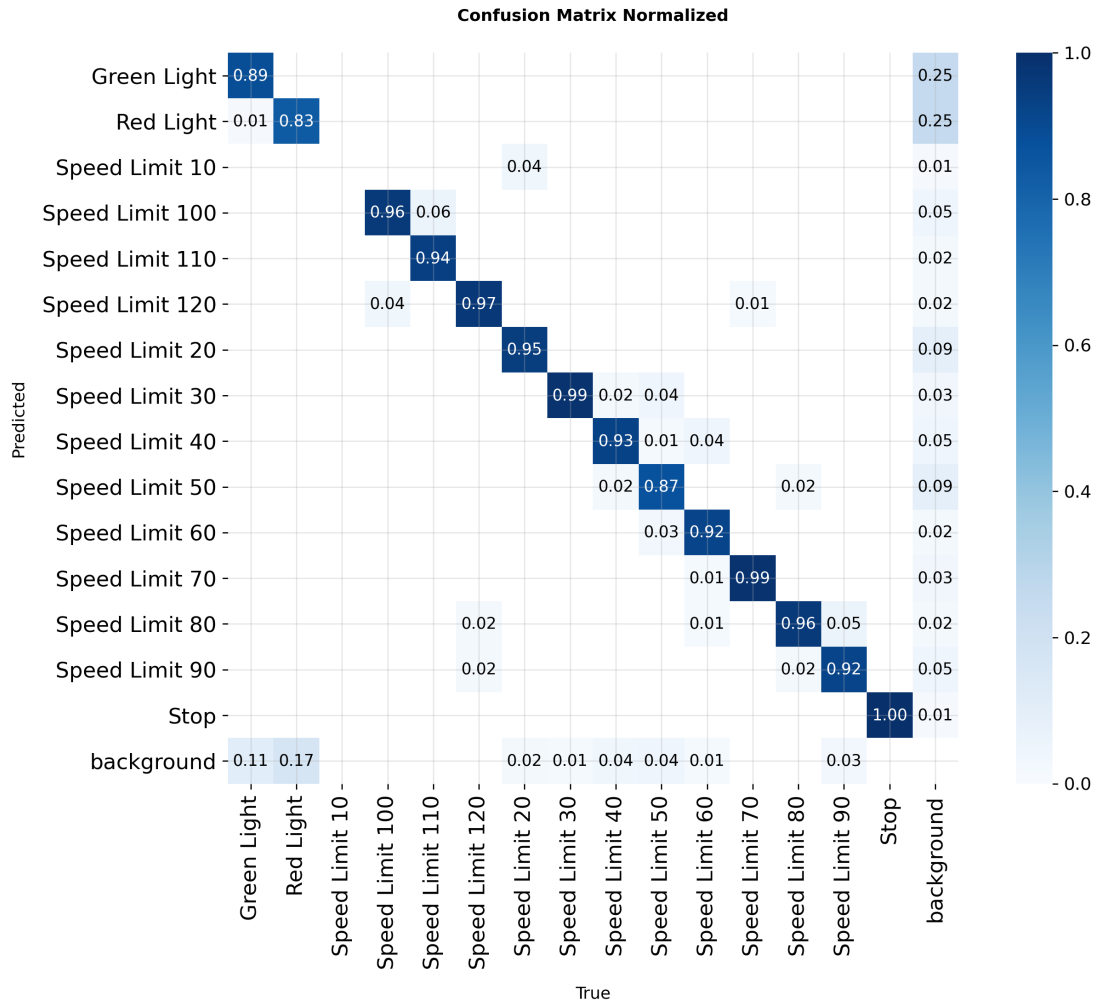
Cần nhấn mạnh rằng *tập kiểm tra không phát hiện được vấn đề này*, bởi nó có cùng

phân bố với tập huấn luyện. Đây chính là dạng thất bại mà tài liệu về thiên lệch bộ dữ liệu đã cảnh báo [11, 27]: chỉ số cao trên tập kiểm tra cùng phân bố không bảo đảm năng lực trên dữ liệu thật.

Một yếu tố thứ hai làm chỉ số bị thổi phồng: thủ tục kiểm toán rò rỉ phát hiện **155 nhóm ảnh nguồn xuất hiện ở nhiều hơn một tập**, trong đó **101 nhóm trùng byte hoàn toàn**. Quy về tập kiểm tra, có **65 trong số 638 ảnh có bản sao trong tập huấn luyện**, tức 10,2% bị rò rỉ, chỉ còn **573 ảnh thực sự sạch**. Hiện tượng trùng lặp gần giữa các tập là một nguồn sai lệch đã được ghi nhận trên nhiều bộ chuẩn phổ biến [36]: mô hình đã nhìn thấy đúng những ảnh đó trong lúc huấn luyện, nên phần dự đoán trên chúng thiên về ghi nhớ hơn là khái quát hoá. Con số 0,9539 vì vậy nên được hiểu là cận trên, và mọi so sánh nghiêm túc về sau nên báo cáo thêm một cột đo riêng trên 573 ảnh sạch.

4.6 Phân tích lỗi

4.6.1 Nhầm lẫn giữa các lớp



Hình 4.3. Ma trận nhầm lẫn chuẩn hoá của YOLOv8n trên tập kiểm định

Ma trận nhầm lẫn ở Hình 4.3 cho thấy đường chéo chính chiếm ưu thế rõ rệt. Các nhầm lẫn còn lại tập trung vào nhóm biển giới hạn tốc độ. Điều này hợp lý về mặt thị giác: mười ba lớp giới hạn tốc độ có hình dạng, màu sắc và viền hoàn toàn giống nhau, chỉ khác ở chữ số bên trong. Ở độ phân giải thấp, chữ số là chi tiết đầu tiên bị mất, trong khi hình tròn viền đỏ vẫn nhận ra được. Nói cách khác, mô hình *định vị* tốt hơn *phân loại*.

Quan sát này gợi ý một hướng thiết kế: tách bài toán thành hai giai đoạn, một bộ phát hiện không phân biệt lớp để tìm vị trí biển báo, và một bộ phân lớp riêng chạy trên ảnh đã

cắt. Cách tổ chức này cũng cho phép tận dụng đúng mục đích 1.922 ảnh cắt kiểu GTSRB vốn được sinh ra cho bài toán phân lớp.

4.6.2 Ảnh hưởng của mất cân bằng lớp

Lớp *Speed Limit 10* chỉ có 22 mẫu trên toàn bộ dữ liệu, tương đương khoảng 15 mẫu trong tập huấn luyện. Với số lượng này, mô hình khó học được biểu diễn ổn định, và bất kỳ chỉ số nào tính riêng cho lớp đó đều có phương sai rất lớn. Đây là biểu hiện điển hình của bài toán mất cân bằng lớp trong học sâu [8].

Cần lưu ý rằng mAP lấy trung bình *không có trọng số* trên các lớp, nên một lớp thiếu số có *AP* thấp sẽ kéo mAP tổng xuống ngang bằng với một lớp đa số. Việc mô hình vẫn đạt 0,9539 cho thấy các lớp thiếu số không quá kém — nhưng cũng có thể phản ánh việc các mẫu hiếm trong tập kiểm tra tình cờ dễ.



(a) Nhân thật

(b) Dự đoán của mô hình

Hình 4.4. Đối chiếu nhân thật và dự đoán trên một lô ảnh kiểm định

4.7 Bàn luận

Đối chiếu với các công trình đã nêu ở Chương 2, kết quả của đề tài tương đồng ở ba điểm và khác biệt ở một điểm.

Điểm tương đồng thứ nhất là ưu thế của mạng tích chập khi dữ liệu hạn chế. Nhận định rằng transformer cần lượng dữ liệu lớn hơn CNN [18] được tái khẳng định rõ ràng qua

chênh lệch giữa 0,9539 và 0,1220. Điểm tương đồng thứ hai là tốc độ hội tụ chậm của DETR đúng như mô tả trong bài báo gốc [4], và việc D-FINE-Nano — một kiến trúc transformer được thiết kế để hội tụ nhanh — đạt 0,7836 trong cùng ngân sách cũng củng cố cách lý giải này. Điểm tương đồng thứ ba là vật thể nhỏ vẫn là điểm yếu hệ thống, phù hợp với các khảo sát đã công bố [9].

Điểm khác biệt nằm ở nhóm E2. Trong khi các nghiên cứu về lượng tử hoá số nguyên [21] thường báo cáo INT8 vừa nén vừa tăng tốc, thí nghiệm ở đây ghi nhận INT8 *giảm* tốc độ. Sự khác biệt này không mâu thuẫn với lý thuyết mà phản ánh một điều kiện tiên quyết thường bị bỏ qua trong các bài trình bày phổ thông: lợi ích tốc độ của lượng tử hoá số nguyên chỉ hiện thực hoá khi môi trường thực thi có nhân tính toán tương ứng được tối ưu cho phần cứng đích.

Cuối cùng, phát hiện về thành phần bộ dữ liệu ở Mục 3.1.4 là đóng góp mang tính phương pháp luận rõ nhất của đề tài. Nó cho thấy rằng khi một bộ dữ liệu được ghép từ nhiều nguồn có mục đích khác nhau — ở đây là trộn ảnh phân lớp kiểu GTSRB với ảnh phát hiện kiểu cảnh đường — thì chỉ số tổng hợp trên tập kiểm tra có thể che giấu hoàn toàn một lỗ hổng năng lực nghiêm trọng. Việc kiểm tra phân bố kích thước theo từng nguồn dữ liệu, thay vì chỉ trên toàn bộ tập, nên được coi là một bước bắt buộc trước khi diễn giải bất kỳ con số mAP nào.

CHƯƠNG 5 : TỔNG KẾT

5.1 Tóm lược kết quả đạt được

Đề tài đã xây dựng một quy trình hoàn chỉnh cho bài toán phát hiện biển báo giao thông, từ khâu kiểm tra dữ liệu thô cho tới bảng số liệu so sánh cuối cùng, và tiến hành bốn nhóm thực nghiệm trên bộ dữ liệu *pkdarabi/cardetection* gồm 4.969 ảnh với 6.012 hộp bao thuộc 15 lớp.

Đối chiếu với ba câu hỏi nghiên cứu đặt ra ở Chương 1:

Câu hỏi 1.1 về so sánh giữa hai họ kiến trúc đã được trả lời rõ ràng. YOLOv8n đạt mAP@0,5 bằng 0,9539 và mAP@0,5:0,95 bằng 0,8058 trên tập kiểm tra, với 6,25 MB trọng số và độ trễ 8,83 ms mỗi ảnh. DETR-ResNet50 chỉ đạt 0,1220 và 0,0988, đồng thời nặng hơn 26,6 lần và chậm hơn 7,2 lần. Tuy nhiên, kết luận đúng đắn không phải là “kiến trúc transformer kém hơn” mà là “ở ngân sách dữ liệu và tính toán của đề tài này, DETR chưa được huấn luyện đủ”. Năm yếu tố gây nhiễu đã được phân tích ở Mục 4.2.2, trong đó việc nhánh DETR không hề có tăng cường dữ liệu trong khi nhánh YOLO có đầy đủ là yếu tố ít được chú ý nhưng nhiều khả năng nặng ký nhất.

Câu hỏi 1.2 về kiến trúc nhẹ và nén mô hình cho câu trả lời phủ định ở phần lớn các nhánh. Cả ba kiến trúc thay thế đều kém hơn mốc tham chiếu của cùng giai đoạn, trong đó YOLO11n tốt nhất nhưng vẫn thấp hơn 3,81 điểm phần trăm. Ở nhóm nén mô hình, chỉ FP16 là lựa chọn không có nhược điểm: giữ nguyên độ chính xác trong khi giảm khoảng một nửa dung lượng. Lượng tử hoá INT8 giảm cả độ chính xác lẫn tốc độ trên môi trường thử nghiệm. Chứng cất tri thức cho kết quả âm ỉ rõ rệt.

Câu hỏi 1.3 về khoảng cách giữa chỉ số và thực tế là câu hỏi thu được câu trả lời có giá trị nhất. Phân tích thành phần bộ dữ liệu cho thấy 38,68% số ảnh là ảnh cắt cận cảnh kiểu GTSRB với cạnh biển chiếm 66,8% cạnh ảnh, trong khi 12,98% là ảnh camera mắt cá với 99,6% vật thể thuộc nhóm nhỏ. Phân bố kích thước vì vậy bị lưỡng cực, thiếu hẳn vùng trung gian tương ứng với biển báo ở khoảng cách lái xe thực tế. Con số 0,9539 do đó phản ánh năng lực trên phân bố của tập kiểm tra chứ không phản ánh năng lực trên cảnh đường phố thật.

5.2 Đóng góp

Ba đóng góp chính có thể rút ra.

Thứ nhất, về mặt kỹ thuật, đề tài cung cấp một quy trình tái lập được với bộ sinh số ngẫu nhiên được gieo hạt tập trung, đường dẫn quản lý ở một nơi duy nhất, và bộ kiểm thử tự động gồm 43 trường hợp chạy trên CPU trong khoảng một giây mà không cần GPU hay tải mô hình.

Thứ hai, về mặt thực nghiệm, bảng so sánh mười cấu hình trên ba trục độ chính xác, tốc độ và dung lượng cho phép người đọc thấy được sự đánh đổi thay vì chỉ một con số duy nhất.

Thứ ba, và quan trọng nhất, đề tài đóng góp một phân tích chỉ ra rằng chỉ số mAP cao trên tập kiểm tra có thể che giấu hoàn toàn một lỗ hổng năng lực, khi bộ dữ liệu được ghép từ nhiều nguồn có mục đích khác nhau. Việc kiểm tra phân bố kích thước vật thể *theo từng nguồn* thay vì chỉ trên toàn tập nên được coi là bước bắt buộc trước khi diễn giải kết quả.

5.3 Hạn chế

Đề tài có những hạn chế sau, được nêu đầy đủ để người đọc đánh giá đúng phạm vi giá trị của kết quả.

Hai nhánh dùng hai cách tính mAP khác nhau. Nhánh YOLO dùng bộ đánh giá của Ultralytics, nhánh DETR dùng thư viện torchmetrics. Hai cài đặt có thể khác nhau ở cách nội suy đường cong PR và cách xử lý ngưỡng IoU, nên chênh lệch giữa hai con số nên được đọc theo xu hướng chứ không theo từng chữ số. Cách khắc phục triệt để là đưa cả hai về cùng một bộ đánh giá COCO.

Bộ dữ liệu có rò rỉ giữa các tập. Khảo sát ghi nhận 65 trong số 638 ảnh kiểm tra có bản sao trong tập huấn luyện, tức 10,2%. Con số 0,9539 vì vậy nên được hiểu là cận trên.

Dung lượng mô hình được đo theo hai định nghĩa khác nhau. Với YOLO, dung lượng lấy theo kích thước tệp trọng số trên đĩa, vốn được lưu ở độ chính xác 16 bit. Với DETR, dung lượng tính bằng tổng số byte của tham số ở độ chính xác 32 bit. Tỷ số 26,6 lần vì vậy phóng đại khoảng cách thực về số lượng tham số, vốn chỉ khoảng 13 lần.

Kết quả đến từ hai giai đoạn trên hai cấu hình phần cứng khác nhau. Nhóm E0 thuộc giai đoạn 1, các nhóm E1, E2 và E3 thuộc giai đoạn 2. Cấu hình YOLOv8n xuất hiện ở cả hai và cho hai bộ số khác nhau (0,9539 và 113,2 khung hình mỗi giây so với 0,9633 và 65,3). Báo cáo đã xử lý bằng cách gắn nhãn giai đoạn cho từng dòng và chỉ so sánh trong nội bộ từng giai đoạn, nhưng đây vẫn là một hạn chế thực sự: nó khiến không thể đặt DETR và các kiến trúc ở nhóm E1 lên cùng một thước đo. Cách khắc phục là chạy lại toàn bộ mười

cấu hình trong một lượt duy nhất trên cùng một máy.

Nhánh DETR không ghi Precision và Recall. Mô-đun đánh giá DETR chỉ xuất mAP, kích thước mô hình và tốc độ, nên Bảng so sánh E0 để trống hai ô này. Đây là thiếu sót của quy trình chứ không phải giới hạn của phương pháp, và sửa được bằng vài dòng mã.

Phép đo tốc độ chưa hoàn toàn công bằng. Nhánh YOLO được đo qua hàm dự đoán đầy đủ, bao gồm tiền xử lý và NMS, trong khi nhánh DETR chỉ đo lượt truyền xuôi thuần túy trên đầu vào đã nằm sẵn trên thiết bị. Chiều thiên lệch này có lợi cho DETR, nên kết luận rằng YOLO nhanh hơn vẫn giữ nguyên giá trị, thậm chí là một ước lượng dè dặt.

Tập lớp bị giới hạn. Mười lăm lớp chỉ gồm đèn tín hiệu và biển giới hạn tốc độ. Các nhóm biển cấm, biển nguy hiểm và biển chỉ dẫn hoàn toàn không có trong tập nhãn, nên mô hình không thể phát hiện chúng dù chất lượng huấn luyện có tốt đến đâu.

5.4 Hướng phát triển

Các hướng phát triển được sắp theo thứ tự ưu tiên, từ chi phí thấp tới chi phí cao.

Ưu tiên 1 — Thống nhất bộ đánh giá cho toàn bộ thực nghiệm. Nhóm E3 đã dùng pycocotools và nhờ đó có *AP* tách theo ba nhóm kích thước — chỉ số cho thấy mọi khác biệt giữa các cấu hình đều nằm ở vật thể nhỏ. Áp dụng cùng bộ đánh giá đó cho các nhóm E0, E1 và E2 sẽ vừa bổ sung chỉ số này cho toàn bộ mười cấu hình, vừa khắc phục hạn chế về hai cách tính mAP khác nhau giữa nhánh YOLO và nhánh DETR. Đây là việc dễ nhất trong danh sách vì mã đánh giá đã có sẵn.

Ưu tiên 2 — Làm sạch và cân bằng lại bộ dữ liệu. Loại bỏ 65 ảnh kiểm tra bị trùng, sau đó thử huấn luyện lại khi đã loại hoặc giảm mạnh nhóm 1.922 ảnh cắt kiểu GTSRB. Giả thuyết là mAP tổng sẽ giảm nhưng *AP* trên nhóm vật thể nhỏ và hiệu năng trên video thật sẽ tăng — bản thân việc kiểm chứng giả thuyết này đã là một thí nghiệm có giá trị.

Ưu tiên 3 — Điều chỉnh kiến trúc cho vật thể nhỏ. Huấn luyện ở kích thước ảnh 960 hoặc 1280 thay vì 640, và bổ sung đầu dự đoán P2 hoạt động ở bước sải 4 thay vì 8. Đây là biện pháp dễ hơn nhiều so với chuyển sang một mô hình lớn hơn.

Ưu tiên 4 — Tách bài toán thành hai giai đoạn. Một bộ phát hiện không phân biệt lớp để định vị biển báo, kết hợp một bộ phân lớp riêng chạy trên ảnh đã cắt. Cách tổ chức này giải quyết trực tiếp hiện tượng nhầm lẫn giữa các biển giới hạn tốc độ đã quan sát ở ma trận nhầm lẫn, đồng thời tận dụng đúng mục đích của 1.922 ảnh cắt kiểu GTSRB.

Ưu tiên 5 — Bổ sung dữ liệu đúng miền. Không kỹ thuật tăng cường nào thay thế

được dữ liệu thật đúng miền. Gán nhãn vài trăm khung hình từ camera hành trình trong điều kiện giao thông mục tiêu rồi tinh chỉnh sẽ có tác động lớn hơn mọi biện pháp nêu trên. Quy trình tiết kiệm nhất là dùng mô hình hiện tại sinh nhãn giả ở ngưỡng tin cậy thấp, sửa tay, rồi tinh chỉnh. Nếu cần bổ sung dữ liệu công khai, TT100K [25] và Mapillary Traffic Sign [26] là hai lựa chọn phù hợp vì đều là ảnh cảnh đường thật.

Ưu tiên 6 — Huấn luyện DETR trong điều kiện công bằng. Tăng số chu kỳ lên gấp mười tới ba mươi lần, tăng kích thước lô hiệu dụng bằng cách tích lũy gradient, bổ sung tăng cường dữ liệu tương đương nhánh YOLO, và cân nhắc dùng Deformable DETR [19] vốn hội tụ nhanh hơn đáng kể. Chỉ khi đó mới có thể kết luận điều gì về so sánh giữa hai họ kiến trúc.

TÀI LIỆU THAM KHẢO

- [1] S. Ren, K. He, R. Girshick, and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, 2017.
- [2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779–788.
- [3] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg, “SSD: Single shot multibox detector,” in *European Conference on Computer Vision (ECCV)*, 2016, pp. 21–37.
- [4] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with transformers,” in *European Conference on Computer Vision (ECCV)*, 2020, pp. 213–229.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [6] H. W. Kuhn, “The Hungarian method for the assignment problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.
- [7] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft COCO: Common objects in context,” in *European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755.
- [8] J. M. Johnson and T. M. Khoshgoftaar, “Survey on deep learning with class imbalance,” *Journal of Big Data*, vol. 6, no. 1, p. 27, 2019.
- [9] K. Tong, Y. Wu, and F. Zhou, “Recent advances in small object detection based on deep learning: A review,” *Image and Vision Computing*, vol. 97, p. 103910, 2020.
- [10] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, “Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition,” *Neural Networks*, vol. 32, pp. 323–332, 2012.

- [11] A. Torralba and A. A. Efros, “Unbiased look at dataset bias,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011, pp. 1521–1528.
- [12] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The PASCAL visual object classes (VOC) challenge,” *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.
- [13] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.
- [14] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, “Feature pyramid networks for object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 2117–2125.
- [15] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” <https://github.com/ultralytics/ultralytics>, 2023.
- [16] Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye, and D. Ren, “Distance-IoU loss: Faster and better learning for bounding box regression,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, 2020, pp. 12 993–13 000.
- [17] X. Li, W. Wang, L. Wu, S. Chen, X. Hu, J. Li, J. Tang, and J. Yang, “Generalized focal loss: Learning qualified and distributed bounding boxes for dense object detection,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 21 002–21 012, 2020.
- [18] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [19] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, “Deformable DETR: Deformable transformers for end-to-end object detection,” in *International Conference on Learning Representations (ICLR)*, 2021.

- [20] Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang, Q. Dang, Y. Liu, and J. Chen, “DETRs beat YOLOs on real-time object detection,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024, pp. 16 965–16 974.
- [21] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.
- [22] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, “Mixed precision training,” in *International Conference on Learning Representations (ICLR)*, 2018.
- [23] G. Hinton, O. Vinyals, and J. Dean, “Distilling the knowledge in a neural network,” *arXiv preprint arXiv:1503.02531*, 2015.
- [24] S. Houben, J. Stallkamp, J. Salmen, M. Schlipsing, and C. Igel, “Detection of traffic signs in real-world images: The German traffic sign detection benchmark,” in *International Joint Conference on Neural Networks (IJCNN)*, 2013, pp. 1–8.
- [25] Z. Zhu, D. Liang, S. Zhang, X. Huang, B. Li, and S. Hu, “Traffic-sign detection and classification in the wild,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 2110–2118.
- [26] C. Ertler, J. Mislej, T. Ollmann, L. Porzi, G. Neuhold, and Y. Kuang, “The Mapillary traffic sign dataset for detection and classification on a global scale,” in *European Conference on Computer Vision (ECCV)*, 2020, pp. 68–84.
- [27] B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, “Do ImageNet classifiers generalize to ImageNet?” in *International Conference on Machine Learning (ICML)*, 2019, pp. 5389–5400.
- [28] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.

- [29] C. Shorten and T. M. Khoshgoftaar, “A survey on image data augmentation for deep learning,” *Journal of Big Data*, vol. 6, no. 1, p. 60, 2019.
- [30] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, “YOLOv4: Optimal speed and accuracy of object detection,” *arXiv preprint arXiv:2004.10934*, 2020.
- [31] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz *et al.*, “Transformers: State-of-the-art natural language processing,” in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020, pp. 38–45.
- [32] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [33] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” in *International Conference on Machine Learning (ICML)*, 2013, pp. 1310–1318.
- [34] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [35] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, “Searching for MobileNetV3,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1314–1324.
- [36] B. Barz and J. Denzler, “Do we train on test data? purging CIFAR of near-duplicates,” *Journal of Imaging*, vol. 6, no. 6, p. 41, 2020.

PHỤ LỤC

Phụ lục liệt kê tài nguyên và chi tiết kỹ thuật phục vụ việc tái lập kết quả đề tài.

A. Tài nguyên công khai

Bảng .1. Tài nguyên sử dụng trong đề tài

Tài nguyên	Địa chỉ / định danh
Bộ dữ liệu	https://www.kaggle.com/datasets/pkdarabi/cardetection
Trọng số YOLOv8n	yolov8n.pt — Ultralytics, tiền huấn luyện trên COCO
Trọng số DETR	facebook/detr-resnet-50 — Hugging Face Hub
Thư viện phát hiện	Ultralytics YOLO [15]
Thư viện transformer	Hugging Face Transformers [31]
Nền tảng học sâu	PyTorch [28]
Môi trường huấn luyện	Google Colab, GPU đơn

B. Danh sách lớp

Mười lăm lớp được giữ nguyên theo bộ dữ liệu gốc, không gộp và không ánh xạ lại:

Bảng .2. Danh sách 15 lớp và mã tương ứng

Mã	Tên lớp	Mã	Tên lớp
0	Green Light	8	Speed Limit 40
1	Red Light	9	Speed Limit 50
2	Speed Limit 10	10	Speed Limit 60
3	Speed Limit 100	11	Speed Limit 70
4	Speed Limit 110	12	Speed Limit 80
5	Speed Limit 120	13	Speed Limit 90
6	Speed Limit 20	14	Stop
7	Speed Limit 30		

C. Cách chạy lại quy trình

Mỗi bước của quy trình là một mô-đun độc lập, gọi được trực tiếp từ dòng lệnh:

```

1 # 1. Kiểm tra dữ liệu và đóng gói tệp cấu hình
2 python -m src.data.inspect_dataset --write-configs
3

```

```

4 # 2. Kiểm tra chất lượng nhãn
5 python -m src.data.validate_labels
6
7 # 3. Phân tích khám phá dữ liệu
8 python -m src.eda.class_distribution
9 python -m src.eda.bbox_statistics
10 python -m src.eda.image_statistics
11 python -m src.eda.heatmap_analysis
12
13 # 4. Huấn luyện và đánh giá nhanh YOLO (cần GPU)
14 python -m src.training.train_yolo --model yolov8n.pt --epochs 30
15 python -m src.evaluation.evaluate_yolo --split test
16
17 # 5. Chuyển sang COCO, huấn luyện và đánh giá nhanh DETR (cần GPU)
18 python -m src.data.convert_to_coco
19 python -m src.training.train_detr
20 python -m src.evaluation.evaluate_detr --split test
21
22 # 6. Tổng hợp bảng so sánh
23 python -m src.evaluation.compare_models
24
25 # 7. Chạy bộ kiểm thử tự động (chỉ cần CPU, khoảng 1 giây)
26 pytest

```

Listing 1. Các lệnh chạy lại từng bước của quy trình

D. Bảng ánh xạ hình vẽ và nguồn dữ liệu

Mọi hình và bảng số liệu trong báo cáo đều được sinh ra từ dữ liệu thật, không có số liệu minh họa. Bảng .3 chỉ rõ nguồn của từng nhóm.

Bảng .3. Ánh xạ giữa nội dung báo cáo và tệp kết quả sinh ra bởi quy trình

Nội dung trong báo cáo	Nguồn
Bảng thống kê tổng quan	results/tables/dataset_summary.csv
Bảng và hình phân bố lớp	results/tables/class_distribution.csv
Bảng phân nhóm kích thước	results/tables/bbox_size_categories.csv
Thống kê ảnh	results/tables/image_statistics.csv
Báo cáo chất lượng nhãn	results/tables/data_quality_report.csv
Chỉ số YOLO	results/metrics/yolo_baseline.json
Chỉ số DETR	results/metrics/detr_baseline.json
Đường cong huấn luyện, PR, F_1 , ma trận nhầm lẫn	Thư mục chạy của Ultralytics
Bảng thành phần bộ dữ liệu	Phân tích bổ sung theo quy ước đặt tên tệp
Bảng và số liệu nhóm E3	notebooks/giaidoan2.ipynb (đánh giá bằng pycocotools)

E. Ghi chú về tính tái lập

Hạt ngẫu nhiên mặc định là 42, được đặt cho biến môi trường băm của Python, thư viện ngẫu nhiên chuẩn, NumPy, PyTorch và PyTorch CUDA. Chế độ tắt định của cuDNN được bật và chế độ tự dò thuật toán nhanh nhất được tắt — điều này làm chậm huấn luyện một chút nhưng là điều kiện cần để kết quả lặp lại được giữa các lần chạy.

Cần lưu ý rằng ngay cả khi đã gieo hạt đầy đủ, kết quả vẫn có thể sai khác nhỏ giữa các phiên bản thư viện, các kiến trúc GPU khác nhau và giữa chế độ CPU với GPU. Các con số trong báo cáo được đo trên môi trường Google Colab với GPU đơn tại thời điểm thực hiện đề tài.